

Σύστημα Ολοκληρωμένης Διαχείρισης Ιατρικών Ραντεβού και Υπηρεσιών Υγείας

ΧΡΗΣΤΟΣ ΤΡΙΑΝΤΑΦΥΛΛΑΚΗΣ

ΝΙΚΟΛΑΟΣ ΧΡΙΣΤΟΠΟΥΛΟΣ

ΧΡΗΣΤΟΣ ΣΤΟΥΡΝΑΡΑΣ

Η εργασία επικεντρώνεται στη δημιουργία μιας σύγχρονης και λειτουργικής βάσης δεδομένων, η οποία θα διασυνδέει αποτελεσματικά γιατρούς και ασθενείς. Η πλατφόρμα στοχεύει να διευκολύνει τη διαχείριση ιατρικών ραντεβού και υπηρεσιών, παρέχοντας μια ολοκληρωμένη λύση για την οργάνωση και την επικοινωνία στον τομέα της υγείας

Πανεπιστήμιο Αιγαίου
Τμήμα Μηχανικών Πληροφοριακών και Επικοινωνιακών Συστημάτων

ΒΑΣΕΙΣ ΔΕΔΟΜΕΝΩΝ ΙΙ

Σύστημα Ολοκληρωμένης Διαχείρισης Ιατρικών Ραντεβού και Υπηρεσιών Υγείας

Η εργασία πραγματοποιήθηκε από τους:

- icsd22219: Χρήστος Τριανταφυλλάκης
- icsd22250: Νικόλαος Χριστόπουλος
- icsd22202: Χρήστος Στουρνάρας

Διδάσκοντας: Παναγιώτης Συμεωνίδης

Εργαστηριακός Διδάσκοντας: Αλέξανδρος Φακής

Σάμος, 31/1/2025

Περιεχόμενα

1. **Εισαγωγή περιγραφή εργασίας**
2. **Απαιτήσεις συστήματος**
   - 2.1. Λειτουργικές απαιτήσεις
   - 2.2. Μη-Λειτουργικές απαιτήσεις
3. **Αποθήκευση δεδομένων στην βάση**
   - 3.1. Αποθήκευση για κάθε αντικείμενο στον μικρόκοσμο της εφαρμογής
4. **Ανάκτηση δεδομένων**
   - 4.1. Δεδομένα που μπορούν να αλλάξουν από τον χρήστη
   - 4.2. Ερωτήματα SQL (απλά και σύνθετα)
5. **Διάγραμμα οντοτήτων συσχετίσεων**
6. **Σχεσιακό σχήμα**
7. **Δημιουργία βάσης και εισαγωγή δεδομένων στους πίνακες**
8. **Ανάκτηση δεδομένων από την βάση**
9. **Υλοποίηση εφαρμογής**
10. **Υλοποίηση Βάσης Γραφοδεδομένων**
11. **Το μοντέλο γραφοδεδομένων**
12. **Ο CYPHER κώδικας για την δημιουργία της βάσης**
13. **Ο CYPHER κώδικας κώδικα μαζί με screenshot αποτελεσμάτων για τα ερωτήματα που απαντήθηκαν**
14. **Συμπέρασμα**

1. Περιγραφή σεναρίου

Η ομάδα μας έχει στόχο να κατασκευάσει ένα σύστημα διαχείρισης ιατρικών δεδομένων, το οποίο συνδέει ασθενείς, γιατρούς, εξετάσεις, συνταγές και άλλες σχετικές πληροφορίες. Κάθε ασθενής έχει μοναδικό αναγνωριστικό (Patient_ID) και αποθηκεύονται πληροφορίες όπως το όνομα, η ημερομηνία γέννησης, τα στοιχεία επικοινωνίας και η διεύθυνσή του. Επιπλέον, κάθε ασθενής διαθέτει ένα ιατρικό ιστορικό (Medical_History) που περιλαμβάνει σημαντικά δεδομένα, όπως σημειώσεις, αλλεργίες και φάρμακα. Το ιστορικό αυτό συνδέεται με καταστάσεις (Condition), που μπορεί να είναι ασθένειες ή διαγνώσεις και καταγράφονται με πληροφορίες για τον τύπο της ασθένειας και την κατάστασή της. Το σύστημα υποστηρίζει τη διαδικασία συνταγογράφησης, καθώς οι ασθενείς λαμβάνουν συνταγές (Prescription) που περιέχουν φάρμακα, τη συχνότητα λήψης και τη διάρκεια θεραπείας. Κάθε συνταγή συνδέεται υποχρεωτικά με μία ή περισσότερες ιατρικές εξετάσεις (Medical_test), οι οποίες καταγράφουν πληροφορίες όπως η ημερομηνία, ο τύπος της εξέτασης, τα αποτελέσματα και σημειώσεις. Ωστόσο, κάθε εξέταση μπορεί να πραγματοποιηθεί χωρίς να απαιτείται να βασίζεται σε συνταγή, ενώ κάθε συνταγή πρέπει να σχετίζεται τουλάχιστον με μία εξέταση. Οι γιατροί, οι οποίοι αναγνωρίζονται από το μοναδικό τους αναγνωριστικό (Doctor_ID), διαθέτουν πληροφορίες όπως το όνομα, την ειδικότητά τους και τα στοιχεία επικοινωνίας τους. Είναι υπεύθυνοι για τη συνταγογράφηση φαρμάκων και τη δημιουργία συνταγών, ενώ συνδέονται επίσης με τα ραντεβού που λαμβάνουν από τους ασθενείς. Οι ασθενείς έχουν τη δυνατότητα να κλείσουν ραντεβού (Appointment) με γιατρούς μέσω του συστήματος. Τα ραντεβού περιλαμβάνουν λεπτομέρειες όπως την ημερομηνία, την τοποθεσία και τον τύπο της υπηρεσίας που προσφέρεται. Μετά την ολοκλήρωση ενός ραντεβού, οι ασθενείς μπορούν να αφήσουν αξιολογήσεις (Review) για την εμπειρία τους, παρέχοντας βαθμολογία και σχόλια που συνδέονται με τις υπηρεσίες ή τις εξετάσεις που έλαβαν. Το σύστημα επίσης περιλαμβάνει τη δυνατότητα διαχείρισης πληρωμών (Payment). Κάθε πληρωμή καταγράφεται με λεπτομέρειες όπως το ποσό, τη μέθοδο πληρωμής, την ημερομηνία και την κατάσταση πληρωμής. Οι πληρωμές σχετίζονται με τα ραντεβού ή τις υπηρεσίες που λαμβάνει ο ασθενής, διασφαλίζοντας τη διαφάνεια στις οικονομικές συναλλαγές. Συνολικά, το σύστημα παρέχει ένα ολοκληρωμένο περιβάλλον για τη διαχείριση ιατρικών υπηρεσιών, συνδέοντας τις διάφορες πτυχές της φροντίδας ασθενών, από τη διάγνωση και τη θεραπεία έως την παρακολούθηση και την πληρωμή. Επιτρέπει την αποτελεσματική επικοινωνία μεταξύ γιατρών και ασθενών, διατηρώντας όλα τα δεδομένα οργανωμένα και προσβάσιμα, ενώ προάγει τη διαφάνεια και τη βελτίωση της ποιότητας των υπηρεσιών υγείας.

2. Απαιτήσεις συστήματος

Διαχείριση Ασθενών:

Αποθήκευση των στοιχείων ασθενών, όπως όνομα, επώνυμο, ημερομηνία γέννησης, στοιχεία επικοινωνίας (διεύθυνση, email, τηλέφωνο) και ιατρικό ιστορικό (π.χ., αλλεργίες, φάρμακα, παθήσεις, σημειώσεις).

Διαχείριση Γιατρών:

Καταγραφή στοιχείων γιατρών, όπως όνομα, επώνυμο, ειδικότητα, ημερομηνία γέννησης και στοιχεία επικοινωνίας (τηλέφωνο, email, διεύθυνση).

Διαχείριση Ραντεβού:

Δημιουργία, τροποποίηση και διαγραφή ραντεβού.

Σύνδεση των ραντεβού με ασθενείς και γιατρούς.

Εισαγωγή πληροφοριών ραντεβού, όπως ημερομηνία, ώρα, τοποθεσία, είδος υπηρεσίας, σημειώσεις και κατάσταση.

Διαχείριση Ιατρικών Συνταγών:

Καταχώρηση συνταγών που γράφονται από γιατρούς για ασθενείς.

Εισαγωγή πληροφοριών όπως το φάρμακο, η δοσολογία, η συχνότητα, η διάρκεια και σημειώσεις.

Σύνδεση της συνταγής με ένα συγκεκριμένο ραντεβού, γιατρό και ασθενή.

Διαχείριση Ιατρικών Εξετάσεων:

Καταχώρηση αποτελεσμάτων ιατρικών εξετάσεων.Καταγραφή πληροφοριών τους είδος εξέτασης, ημερομηνία, αποτελέσματα και σημειώσεις.

Σύνδεση των εξετάσεων με συνταγές και ραντεβού.

Διαχείριση Πληρωμών:

Καταγραφή πληρωμών που γίνονται από τους ασθενείς.

Εισαγωγή πληροφοριών τους ποσό, ημερομηνία, τρόπος πληρωμής, κατάσταση και σύνδεση με ραντεβού, γιατρούς και ασθενείς.

Διαχείριση Ιατρικού Ιστορικού:

Εισαγωγή πληροφοριών για τους παθήσεις, τους αλλεργίες, τα φάρμακα που λαμβάνει τους ασθενής και σημειώσεις σχετικές με το ιατρικό του ιστορικό.

Διαχείριση Αξιολογήσεων:

Καταχώρηση αξιολογήσεων από ασθενείς τους γιατρούς.

Εισαγωγή βαθμολογιών, σημειώσεων και άλλων σχολίων.

2.1 Λειτουργικές Απαιτήσεις

Οι γιατροί πρέπει να μπορούν να:

- Δημιουργούν, επεξεργάζονται και να βλέπουν ραντεβού.
- Συνταγογραφούν φάρμακα σε ασθενείς.
- Βλέπουν το ιστορικό των ασθενών τους.

Οι ασθενείς πρέπει να μπορούν να:

- Κλείνουν ραντεβού με γιατρούς.
- Λαμβάνουν συνταγές και αποτελέσματα εξετάσεων.
- Βλέπουν το ιατρικό τους ιστορικό.

Το σύστημα πρέπει να υποστηρίζει τη δημιουργία αναφορών για:

- Πληρωμές που έχουν γίνει.
- Ραντεβού που πραγματοποιήθηκαν.
- Ιατρικές συνταγές και εξετάσεις.
- Οι πληρωμές πρέπει να συνδέονται με τα ραντεβού και να υπολογίζεται η συνολική οφειλή ενός ασθενούς.

2.2 Μη Λειτουργικές Απαιτήσεις

Ασφάλεια Δεδομένων:

- Τα ιατρικά δεδομένα πρέπει να προστατεύονται σύμφωνα με τους κανονισμούς.
- Πρόσβαση σε συγκεκριμένα δεδομένα πρέπει να έχουν μόνο εξουσιοδοτημένοι χρήστες.

Επιδόσεις:

- Το σύστημα πρέπει να είναι σε θέση να χειρίζεται μεγάλο αριθμό χρηστών ταυτόχρονα, ειδικά σε ώρες αιχμής.

Ευχρηστία:

- Το σύστημα πρέπει να είναι φιλικό προς τον χρήστη, με εύκολη πλοήγηση για ασθενείς και γιατρούς.

Επεκτασιμότητα:

- Το σύστημα πρέπει να επιτρέπει την προσθήκη νέων λειτουργιών στο μέλλον, όπως τηλεϊατρικές υπηρεσίες ή υπενθυμίσεις ραντεβού μέσω email/SMS.

Αναφορές και Ιστορικό:

- Δυνατότητα εξαγωγής δεδομένων για στατιστική ανάλυση ή αρχειοθέτηση.

3. Τι θέλει ο χρήστης να αποθηκεύει στη βάση;

Ο χρήστης θέλει να αποθηκεύει πληροφορίες για:

- **Ασθενείς:** Προσωπικά στοιχεία και το ιατρικό τους ιστορικό.
- **Γιατρούς:** Προσωπικά στοιχεία, ειδικότητες και επικοινωνία.
- **Ραντεβού:** Πληροφορίες για προγραμματισμένα ραντεβού (ημερομηνία, ώρα, τοποθεσία).
- **Συνταγές:** Πληροφορίες για φάρμακα, δοσολογία και οδηγίες χρήσης.
- **Ιατρικές εξετάσεις:** Αποτελέσματα και πληροφορίες για εξετάσεις.

- **Πληρωμές:** Συναλλαγές που πραγματοποιούνται μεταξύ ασθενών και γιατρών.
- **Αξιολογήσεις:** Κριτικές των ασθενών προς τους γιατρούς.

3.1 Τι θα αποθηκεύει για κάθε αντικείμενο στον μικρόκοσμο της εφαρμογής;

Ασθενείς (Patient):

- Patient_ID (Μοναδικό αναγνωριστικό).
- Όνομα, Επώνυμο, Ημερομηνία γέννησης.
- Στοιχεία επικοινωνίας: Τηλέφωνο, Email, Διεύθυνση (πόλη, ταχυδρομικός κώδικας, οδός).

Ιατρικό Ιστορικό (Medical History):

- History_ID (Μοναδικό αναγνωριστικό).
- Παθήσεις, φάρμακα, αλλεργίες, σημειώσεις.

Γιατροί (Doctor):

- Doctor_ID (Μοναδικό αναγνωριστικό).
- Όνομα, Επώνυμο, Ειδικότητα, Ημερομηνία γέννησης.
- Στοιχεία επικοινωνίας: Τηλέφωνο, Email, Διεύθυνση (πόλη, ταχυδρομικός κώδικας, οδός).

Ραντεβού (Appointment):

- Appointment_ID (Μοναδικό αναγνωριστικό).
- Ασθενής (Patient_ID), Γιατρός (Doctor_ID).
- Ημερομηνία, Ώρα, Τοποθεσία, Σημειώσεις, Κατάσταση.
- Είδος υπηρεσίας (π.χ., επίσκεψη, συμβουλευτική).

Ιατρικές Συνταγές (Prescription):

- Prescription_ID (Μοναδικό αναγνωριστικό).
- Ασθενής (Patient_ID), Γιατρός (Doctor_ID), Ραντεβού (Appointment_ID).
- Όνομα φαρμάκου, Δοσολογία, Συχνότητα, Διάρκεια, Σημειώσεις.

Ιατρικές Εξετάσεις (Medical_Test):

- Test_ID (Μοναδικό αναγνωριστικό).
- Σχετίζεται με Ραντεβού (Appointment_ID), Γιατρό (Doctor_ID), και Ασθενή (Patient_ID).
- Ημερομηνία εξέτασης, Είδος εξέτασης, Αποτελέσματα, Σημειώσεις.

Πληρωμές (Payment):

- Payment_ID (Μοναδικό αναγνωριστικό).
- Ασθενής (Patient_ID), Γιατρός (Doctor_ID), Ραντεβού (Appointment_ID).
- Ποσό, Ημερομηνία, Τρόπος πληρωμής (π.χ., κάρτα, μετρητά), Κατάσταση

Αξιολογήσεις (Review):

- Review_ID (Μοναδικό αναγνωριστικό).
- Γιατρός (Doctor_ID), Ασθενής (Patient_ID). Βαθμολογία, Σχόλια.

4. Τι είδους δεδομένα θέλει ο χρήστης να ανακτά;

Πληροφορίες για ασθενείς:

- Προσωπικά στοιχεία (π.χ., όνομα, τηλέφωνο, email).
- Ιατρικό ιστορικό (παθήσεις, φάρμακα, αλλεργίες).
- Ραντεβού του ασθενή (προγραμματισμένα και παρελθοντικά).
- Συνταγές και ιατρικές εξετάσεις που αφορούν τον ασθενή.

Πληροφορίες για γιατρούς:

- Προσωπικά στοιχεία (π.χ., ειδικότητα, τηλέφωνο, email).
- Τα ραντεβού που έχουν με ασθενείς.
- Συνταγές που έχουν γράψει.
- Αξιολογήσεις που έχουν λάβει από ασθενείς.

Πληροφορίες για ραντεβού:

- Ποιος γιατρός και ασθενής συμμετέχουν στο ραντεβού.
- Ημερομηνία, ώρα και τοποθεσία.
- Κατάσταση (π.χ., εκκρεμές, ολοκληρωμένο, ακυρωμένο).

Πληροφορίες για πληρωμές:

- Ποσά που οφείλονται ή έχουν καταβληθεί
- Τρόποι πληρωμής και κατάσταση πληρωμής.

Πληροφορίες για συνταγές και εξετάσεις:

- Λεπτομέρειες συνταγών (π.χ., όνομα φαρμάκου, δοσολογία, διάρκεια).
- Αποτελέσματα ιατρικών εξετάσεων.

4.1 Ποια δεδομένα μπορούν να αλλάξουν από τον χρήστη;

Από τους γιατρούς:

- Τα ραντεβού τους (ημερομηνία, ώρα, τοποθεσία, σημειώσεις).
- Οι συνταγές που γράφουν (π.χ., αλλαγή δοσολογίας ή φαρμάκου).
- Τα αποτελέσματα ιατρικών εξετάσεων (π.χ., ενημέρωση για αποτελέσματα).

Από τους ασθενείς:

- Στοιχεία επικοινωνίας (π.χ., τηλέφωνο, email, διεύθυνση).
- Ραντεβού (π.χ., να ακυρώσουν ή να αλλάξουν την ημερομηνία).

Από τους διαχειριστές του συστήματος:

- Προσθήκη ή διαγραφή ασθενών, γιατρών, ραντεβού, πληρωμών, συνταγών και εξετάσεων. Αλλαγή της κατάστασης πληρωμών (π.χ., από «εκκρεμεί» σε «ολοκληρωμένη»).
- Ενημέρωση στοιχείων γιατρών (π.χ., ειδικότητα ή στοιχεία επικοινωνίας).

4.2 Ερωτήματα (10) απλά και σύνθετα που θα μπορούσαμε να εκτελέσουμε με την χρήση εντολών SELECT.

Απλά ερωτήματα

1.Λίστα ασθενών με τα βασικά στοιχεία τους

2.Όλες οι συνταγές που έχουν γραφτεί για έναν συγκεκριμένο ασθενή:

3.Πληρωμές που έχουν ολοκληρωθεί

4.Λίστα γιατρών με την ειδικότητα και στοιχεία επικοινωνίας

5.Όλες οι εξετάσεις που έχουν πραγματοποιηθεί για έναν συγκεκριμένο ασθενή

Σύνθετα ερωτήματα

6.Λίστα με τα ραντεβού ενός γιατρού σε συγκεκριμένη ημερομηνία

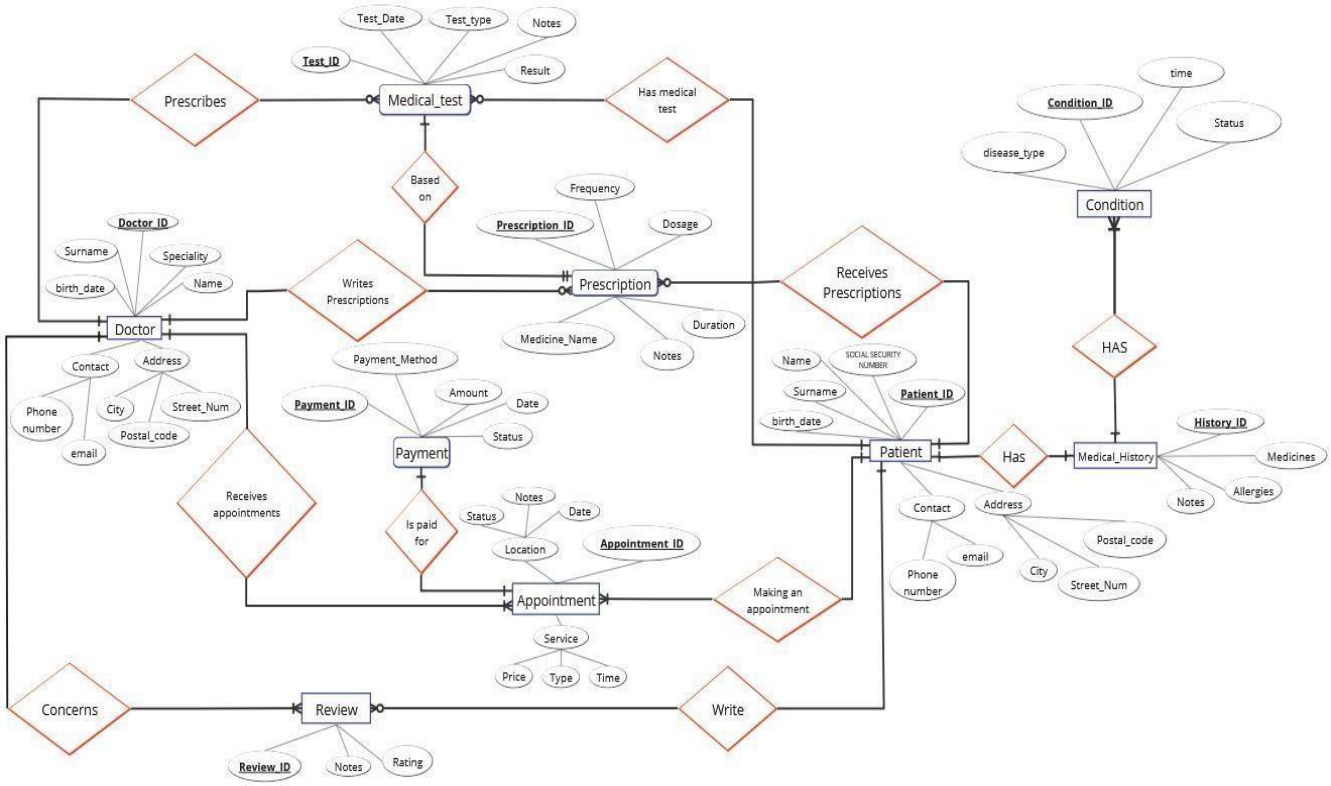
7.Ασθενείς που έχουν επισκεφτεί συγκεκριμένο γιατρό

8.Ποσά που έχουν πληρώσει οι ασθενείς συνολικά

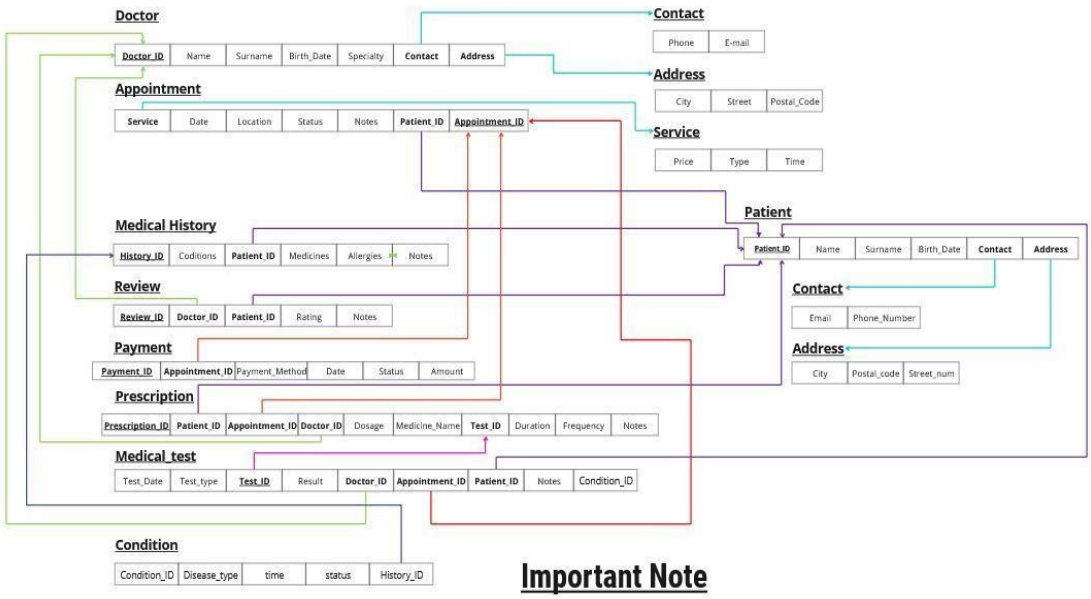
9.Συνταγές και εξετάσεις για έναν συγκεκριμένο ασθενή

10.Γιατροί που έχουν γράψει συνταγές σε συγκεκριμένη χρονική περίοδο

5. Διάγραμμα οντοτήτων συσχετίσεων



6. Σχεσιακό σχήμα



Important Note

The lines with the Doctor_ID key are in colored in green.
The lines with the Patient_ID key are colored in purple.
The lines with the Appointment_ID are colored in red.
The lines with the Test_ID key are colored in magenta.
The secondary table connections are colored in cyan.
The lines with the History_ID are colored in dark blue.
**The use of colors is only for convenience reasons, regarding the correct use and understanding of the diagram above!*
***The primary keys are denoted in bold and underlined writing, while the secondary ones are only in bold.*

7. Δημιουργία βάσης και εισαγωγή δεδομένων στους πίνακες

Βάση δεδομένων: `database_platform`

Δομή πίνακα για τον πίνακα `appointment`

```
CREATE TABLE `appointment` (
  `Appointment_ID` int (11) NOT NULL,
  `Status` text NOT NULL,
  `Notes` text NOT NULL,
  `Date` date NOT NULL,
  `Location` text NOT NULL,
  `Price` float NOT NULL,
  `Time` text NOT NULL,
  `Type` text NOT NULL,
  `Patient_ID` int (11) NOT NULL,
  `Doctor_ID` int (11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων στον πίνακα appointment

```
INSERT INTO `appointment` (`Appointment_ID`, `Status`, `Notes`,
`Date`, `Location`, `Price`, `Time`, `Type`, `Patient_ID`, `Doctor_ID`) VALUES
(1, 'Scheduled', 'Follow-up visit', '2024-12-10', 'Athens', 50, '10:00', 'Consultation', 2, 1),
(2, 'Completed', 'First visit', '2024-12-05', 'Thessaloniki', 70, '12:30', 'Checkup', 3, 2),
(3, 'Scheduled', 'Routine check', '2024-12-15', 'Patras', 30, '14:00', 'Consultation', 4, 3),
(4, 'Cancelled', 'Patient requested cancellation', '2024-12-07', 'Heraklion', 40, '09:00', 'Follow-up', 5, 4),
(5, 'Scheduled', 'MRI referral', '2024-12-20', 'Larisa', 100, '11:15', 'Specialist', 3, 5);
```

Δομή πίνακα για τον πίνακα `conditions`

```
CREATE TABLE `conditions` (
  `Condition_ID` int (11) NOT NULL,
  `time` date NOT NULL,
  `status` text NOT NULL,
  `disease_type` text NOT NULL,
  `History_ID` int (11) NOT NULL  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `conditions`

```
INSERT INTO `conditions` (`Condition_ID`, `time`, `status`,
`disease_type`, `History_ID`) VALUES
(1, '2024-12-01', 'Υπό έλεγχο', 'Υπέρταση', 1),
(2, '2024-11-15', 'Ενεργή', 'Διαβήτης τύπου 2', 2),
(3, '2024-04-20', 'Εποχιακή', 'Αλλεργική Ρινίτιδα', 3),
(4, '2024-03-10', 'Χρόνια', 'Γαστροοισοφαγική Παλινδρόμηση', 4);
```

Δομή πίνακα για τον πίνακα `doctor`

```
CREATE TABLE `doctor` (
  `Doctor_ID` int (11) NOT NULL,
  `Name` text NOT NULL,
  `Surname` text NOT NULL,
  `Specialty` text NOT NULL,
  `Birth_date` date NOT NULL,


`Phone` bigint (20) NOT NULL,
  `Email` text NOT NULL,
```

```
  `City` text NOT NULL,
  `Street` text NOT NULL,
  `Postal_code` int (11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `doctor`

```
INSERT INTO `doctor` (`Doctor_ID`, `Name`, `Surname`, `Specialty`,
`Birth_date`, `Phone`, `Email`, `City`, `Street`, `Postal_code`) VALUES
(1, 'Sophia', 'Papadopoulou', 'Cardiologist', '1980-04-15', 6941234567,
'sophia.pap@example.com', 'Athens', 'Vasilisis Sofias 10', 11521),
(2, 'Dimitris', 'Nikolaidis', 'Orthopedic', '1975-11-20', 6934567890,
'dimitris.niko@example.com', 'Thessaloniki', 'Tsimiski 45', 54623),
(3, 'Eleni', 'Andreadou', 'Pediatrician', '1988-09-12', 6979876543,
'eleni.and@example.com', 'Patras', 'Korinthou 15', 26221),
(4, 'Giorgos', 'Ioannou', 'Dermatologist', '1985-06-05', 6981234789,
'giorgos.io@example.com', 'Heraklion', 'Lydias 32', 71202),
(5, 'Maria', 'Vasileiou', 'Neurologist', '1990-01-18', 6951234987,
'maria.vas@example.com', 'Larisa', 'Kountouriotou 78', 41222);
```

Δομή πίνακα για τον πίνακα `medical_history`

```
CREATE TABLE `medical_history` (
  `History_ID` int (11) NOT NULL,
  `Medicines` text NOT NULL,
  `Notes` text NOT NULL,
  `Allergies` text NOT NULL,
  `Patient_ID` int (11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `medical_history`

```
INSERT INTO `medical_history` (`History_ID`, `Medicines`, `Notes`, `Allergies`, `Patient_ID`) VALUES
(1, 'Paracetamol, Ibuprofen', 'Υψηλή αρτηριακή πίεση, παρακολούθηση.', 'Πενικιλίνη', 2),
(2, 'Metformin', 'Διαβήτης τύπου 2, τακτική λήψη φαρμάκου.', 'Καμία γνωστή αλλεργία', 3),
(3, 'Cetirizine', 'Εποχιακή αλλεργία, αγωγή κατά την άνοιξη.', 'Γύρη', 4),
(4, 'Omeprazole', 'Γαστροοισοφαγική παλινδρόμηση, καθημερινή χρήση.',
'Ασπιρίνη', 5),
(5, 'Amoxicillin', 'Θεραπεία πρόσφατης λοίμωξης, παρακολούθηση.', 'Πενικιλίνη',
1);
```

Δομή πίνακα για τον πίνακα `medical_test`

```
CREATE TABLE `medical_test` (
  `Test_ID` int (11) NOT NULL,
  `Test_date` date NOT NULL,
  `Test_type` text NOT NULL,
  `Notes` text NOT NULL,
  `Result` text NOT NULL,
  `Doctor_ID` int(11) NOT NULL,
  `Appointment_ID` int(11) NOT NULL,
```

```
  `Patient_ID` int(11) NOT NULL


) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `medical_test`

```
INSERT INTO `medical_test` (`Test_ID`, `Test_date`, `Test_type`,
`Notes`, `Result`, `Doctor_ID`, `Appointment_ID`, `Patient_ID`) VALUES
(1, '2024-12-10', 'Blood Test', 'Routine blood test', 'Normal', 1, 1, 2),
(2, '2024-12-05', 'X-Ray', 'X-ray for back pain', 'No issues detected', 2, 2, 3),
(3, '2024-12-15', 'Allergy Test', 'Skin test for allergies', 'Positive for pollen', 3, 3, 4),
(4, '2024-12-07', 'MRI', 'MRI for knee pain', 'Signs of mild arthritis', 4, 4, 5),
(5, '2024-12-20', 'EEG', 'Neurological assessment', 'Normal brain activity', 5, 5,
3);
```

Δομή πίνακα για τον πίνακα `patient`

```
  CREATE TABLE `patient` (
 `Patient_ID` int(11) NOT NULL,
  `Name` text NOT NULL,
  `Surname` text NOT NULL,
  `Social_sec_num` int(11) NOT NULL,
  `birth_date` date NOT NULL,
  `Phone` bigint(20) NOT NULL,
  `Email` text NOT NULL,
  `City` text NOT NULL,
  `Postal_code` int(11) NOT NULL,
  `Street_num` text NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `patient`

```
INSERT INTO `patient` (`Patient_ID`, `Name`, `Surname`,

`Social_sec_num`, `birth_date`, `Phone`, `Email`, `City`, `Postal_code`, `Street_num`) VALUES

(1, 'John', 'Doe', 123456789, '1985-05-12', 6971234567,
'john.doe@example.com', 'Athens', 10435, '15A'),

(2, 'Maria', 'Papadopoulou', 987654321, '1990-08-25', 6987654321,

'maria.pap@example.com', 'Thessaloniki', 54623, '25'),

(3, 'Kostas', 'Giannakos', 456789123, '1978-12-01', 6941237890,
'kostas.giannakos@example.com', 'Patra', 26223, '10'),

(4, 'Eleni', 'Vasileiou', 789123456, '2000-07-15', 6934567890,
'eleni.vas@example.com', 'Heraklion', 71202, '5'),

(5, 'Nikos', 'Andreadis', 159753684, '1982-03-10', 6901234567,
'nikos.and@example.com', 'Larisa', 41222, '20');
```

Δομή πίνακα για τον πίνακα `payment`

```
 CREATE TABLE `payment` (

 `Payment_ID` int(11) NOT NULL,

 `Status` date NOT NULL,

 `Date` date NOT NULL,

 `Amount` float NOT NULL,

 `Payment_method` text NOT NULL,

 `Appointment_ID` int(11) NOT NULL

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `payment`

```
INSERT INTO `payment` (`Payment_ID`, `Status`, `Date`, `Amount`,
`Payment_method`, `Appointment_ID`) VALUES
(1, '0000-00-00', '2024-12-01', 50, 'Credit Card', 1),
(2, '0000-00-00', '2024-12-02', 75, 'Cash', 2),
(3, '0000-00-00', '2024-11-30', 100, 'Debit Card', 3),
(4, '0000-00-00', '2024-11-29', 60, 'Bank Transfer', 4),
(5, '0000-00-00', '2024-12-01', 80, 'Cash', 5);
```

Δομή πίνακα για τον πίνακα `prescription`

```
CREATE TABLE `prescription` (
  `Prescription_ID` int(11) NOT NULL,
  `Frequency` text NOT NULL,
  `Dosage` text NOT NULL,
  `Medicine_name` text NOT NULL,
    `Notes` text NOT NULL,
  `Duration` float NOT NULL,
  `Patient_ID` int(11) NOT NULL,
  `Appointment_ID` int(11) NOT NULL,
  `Doctor_ID` int(11) NOT NULL,
  `Test_ID` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων του πίνακα `prescription`

```
INSERT INTO `prescription` (`Prescription_ID`, `Frequency`, `Dosage`,
`Medicine_name`, `Notes`, `Duration`, `Patient_ID`,
`Appointment_ID`, `Doctor_ID`, `Test_ID`) VALUES
(1, 'Twice daily', '500mg', 'Amoxicillin', 'For throat infection', 7, 2, 1, 1, 1),
(2, 'Once daily', '10mg', 'Loratadine', 'For allergies', 14, 4, 3, 3, 3),
(3, 'Every 8 hours', '200mg', 'Ibuprofen', 'For knee pain', 5, 5, 4, 4, 4),
(4, 'Once daily', '5mg', 'Amlodipine', 'For blood pressure', 30, 3, 2, 2, 2),
(5, 'Three times daily', '250mg', 'Paracetamol', 'For fever and headache', 3, 4, 1,
5, 5);
```

Δομή πίνακα για τον πίνακα `review`

```
CREATE TABLE `review` (
 `Review_ID` int(11) NOT NULL,
 `Notes` text NOT NULL,
 `Rating` float NOT NULL,
 `Doctor_ID` int(11) NOT NULL,
 `Patient_ID` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Εισαγωγή δεδομένων στον πίνακα `review`

```
INSERT INTO `review` (`Review_ID`, `Notes`, `Rating`, `Doctor_ID`, `Patient_ID`) VALUES
(1, 'Εξαιρετικός γιατρός, με βοήθησε πολύ!', 5, 1, 2),
(2, 'Πολύ επαγγελματίας, αλλά υπήρχε μικρή καθυστέρηση στο ραντεβού.',
4.5, 2, 3),
```

(3, 'Κατανοητικός και με εξήγησε πλήρως τη θεραπεία.', 4.8, 3, 4),

(4, 'Ευγενικός, αλλά δεν έμεινα απόλυτα ικανοποιημένος.', 3.5, 4, 5), (5, 'Άριστη εμπειρία, θα τον/την επισκεφτώ ξανά!', 5, 5, 3);

Εμφάνιση των πινάκων με δεδομένα με την βοήθεια του export στο phpMyAdmin

Πίνακας appointment

Appointment_ID	Status	Notes	Date	Location	Price	Time	Type	Patient_ID	Doctor_ID
1	Scheduled	Follow-up visit	2024-12-10	Athens	50	10:00	Consultation	2	1
2	Completed	First visit	2024-12-05	Thessaloniki	70	12:30	Check-up	3	2
3	Scheduled	Routine check	2024-12-15	Patras	30	14:00	Consultation	4	3
4	Cancelled	Patient requested cancellation	2024-12-07	Heraklion	40	09:00	Follow-up	5	4
5	Scheduled	MRI referral	2024-12-20	Larisa	100	11:15	Specialist	3	5

Πίνακας conditions

Condition ID	time	status	disease_type	History_ID
1	2024-12-01	Υπό έλεγχο	Υπέρταση	1
2	2024-11-15	Ενεργή	Διαβήτης τύπου 2	2
3	2024-04-20	Εποχιακή	Αλλεργική Ρινίτιδα	3
4	2024-03-10	Χρόνια	Γαστροοισοφαγική Παλινδρόμηση	4

Πίνακας doctor

Doctor_ID	Name	Surname	Speciality	Birth_date	Phone	Email	City	Street	Postal_code
1	Sophia	Papadopoulou	Cardiologist	1980-04-15	6941234567	sophia.pap@example.com	Athens	Vasilissis Sofias 10	11521
2	Dimitris	Nikolaidis	Orthopedic	1975-11-20	6934567890	dimitris.niko@example.com	Thessaloniki	Tsimiski 45	54623
3	Eleni	Andreadou	Pediatrician	1988-09-12	6979876543	eleni.and@example.com	Patras	Korinthou 15	26221
4	Giorgos	Ioannou	Dermatologist	1985-06-05	6981234789	giorgos.io@example.com	Heraklion	Lydias 32	71202
5	Maria	Vasileiou	Neurologist	1990-01-18	6951234987	maria.vas@example.com	Larisa	Kountouriotou 78	41222

Πίνακας history

History_ID	Medicines	Notes	Allergies	Patient_ID
1	Paracetamol, Ibuprofen	Υψηλή αρτηριακή πίεση, παρακολούθηση.	Πενικιλίνη	2
2	Metformin	Διαβήτης τύπου 2, τακτική λήψη φαρμάκου.	Καμία γνωστή αλλεργία	3
3	Cetirizine	Εποχιακή αλλεργία, αγωγή κατά την άνοιξη.	Γύρη	4
4	Omeprazole	Γαστροοισοφαγική παλινδρόμηση, καθημερινή χρήση.	Ασπιρίνη	5
5	Amoxicillin	Θεραπεία πρόσφατης λοίμωξης, παρακολούθηση.	Πενικιλίνη	1

Πίνακας medical_test

Test_ID	Test_date	Test_type	Notes	Result	Doctor_ID	AppointmentID	Patient_ID
1	2024-12-10	Blood Test	Routine blood test	Normal	1	1	2
2	2024-12-05	X-Ray	X-ray for back pain	No issues detected	2	2	3
3	2024-12-15	Allergy Test	Skin test for allergies	Positive for pollen	3	3	4
4	2024-12-07	MRI	MRI for knee pain	Signs of mild arthritis	4	4	5
5	2024-12-20	EEG	Neurological assessment	Normal brain activity	5	5	3

Πίνακας patient

Patient_ID	Name	Surname	Social_sec_num	birth_date	Phone	Email	City	Postal_code	Street_num
1	John	Doe	123456789	1985-05-12	6971234567	john.doe@example.com	Athens	10435	15A
2	Maria	Papadopoulou	987654321	1990-08-25	6987654321	maria.pap@example.com	Thessaloniki	54623	25
3	Kostas	Giannakos	456789123	1978-12-01	6941237890	kostas.giannakos@example.com	Patra	26223	10
4	Eleni	Vasileiou	789123456	2000-07-15	6934567890	eleni.vas@example.com	Heraklion	71202	5
5	Nikos	Andreadis	159753684	1982-03-10	6901234567	nikos.and@example.com	Larisa	41222	20

Πίνακας payment

Payment_ID	Status	Date	Amount	Payment_method	AppointmentID
1	0000-00-00	2024-12-01	50	Credit Card	1
2	0000-00-00	2024-12-02	75	Cash	2
3	0000-00-00	2024-11-30	100	Debit Card	3
4	0000-00-00	2024-11-29	60	Bank Transfer	4
5	0000-00-00	2024-12-01	80	Cash	5

Πίνακας prescription

Prescription ID	Frequency	Dosage	Medicine_name	Notes	Duration	Patient_ID	Appointment ID	Doctor_ID	Test_ID
1	Twice daily	500mg	Amoxicillin	For throat infection	7	2	1	1	1
2	Once daily	10mg	Loratadine	For allergies	14	4	3	3	3
3	Every 8 hours	200mg	Ibuprofen	For knee pain	5	5	4	4	4
4	Once daily	5mg	Amlodipine	For blood pressure	30	3	2	2	2
5	Three times daily	250mg	Paracetamol	For fever and headache	3	4	1	5	5

Πίνακας review

Review_ID	Notes	Rating	Doctor_ID	Patient_ID
1	Εξαιρετικός γιατρός, με βοήθησε πολύ!	5	1	2
2	Πολύ επαγγελματίας, αλλά υπήρχε μικρή καθυστέρηση στο ραντεβού.	4.5	2	3
3	Κατανοητικός και με εξήγησε πλήρως τη θεραπεία.	4.8	3	4
4	Ευγενικός, αλλά δεν έμεινα απόλυτα ικανοποιημένος.	3.5	4	5
5	Άριστη εμπειρία, θα τον/την επισκεφτώ ξανά!	5	5	3

8. Ανάκτηση δεδομένων από την βάση

Σε αυτό το κεφάλαιο θα αναφερθούμε σε ερωτήσεις που θα εκτελέσουμε στην βάση δεδομένων (άλλες πιο σύνθετες άλλες πιο απλές σύμφωνα με τις προϋποθέσεις που έχει η εργασία μας).Καθώς και την ερμηνεία τους στην φυσική τους γλώσσα. Τέλος θα αναφερθούμε και στα αποτελέσματα που θα έχουν τα ερωτήματα αφού τα εκτελέσουμε στην βάση δεδομένων.

2 ερωτήματα με διαφορετικές Aggregate Functions (avg, count, min, max, sum)

1ο ερώτημα:

Υπολογίστε τον μέσο όρο πληρωμών ανά μέθοδο πληρωμής.

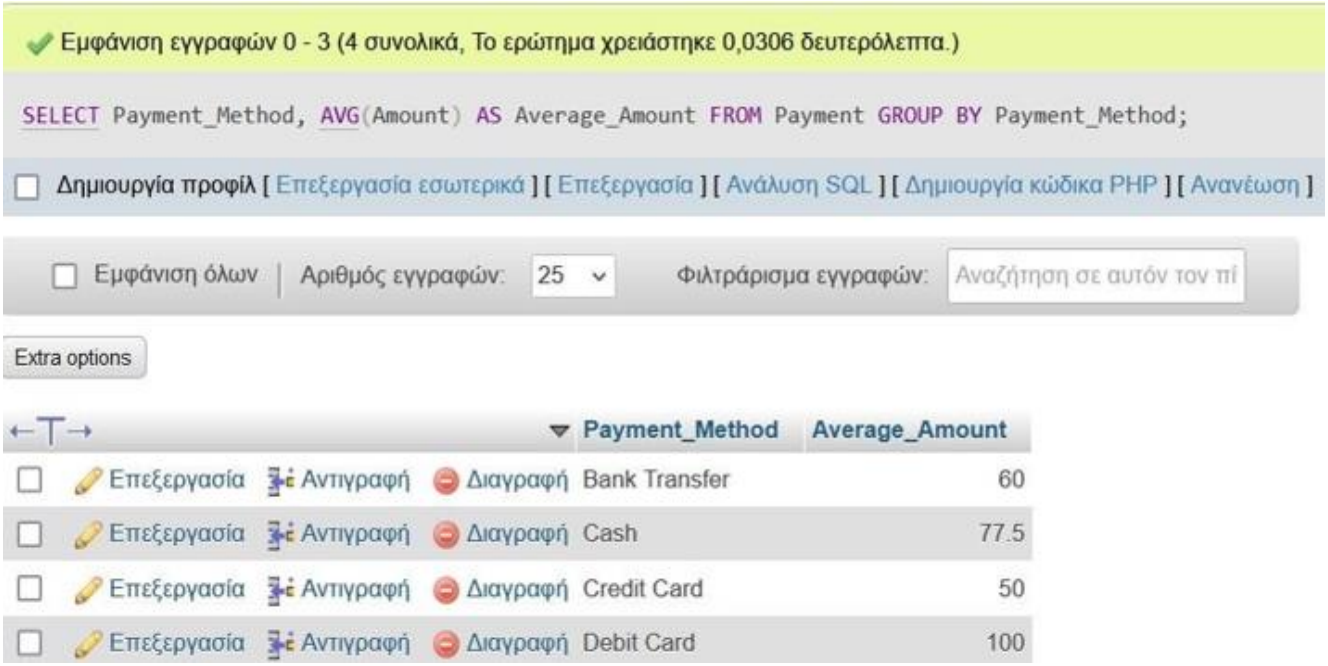
```
SELECT Payment_Method, AVG(Amount) AS Average_Amount

FROM Payment

GROUP BY Payment_Method;
```

Επεξήγηση σε φυσική γλώσσα: Υπολογίζει τον μέσο όρο ποσού που έχει πληρωθεί για κάθε μέθοδο πληρωμής (π.χ., μετρητά, πιστωτική κάρτα).

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



2ο ερώτημα:

Πόσα ραντεβού έχει κάθε γιατρός και ταξινόμησέ τα κατά φθίνουσα σειρά.

```
SELECT D.Doctor_ID, D.Name AS Doctor_Name, COUNT(A.Appointment_ID) AS Total_Appointments

FROM Doctor D

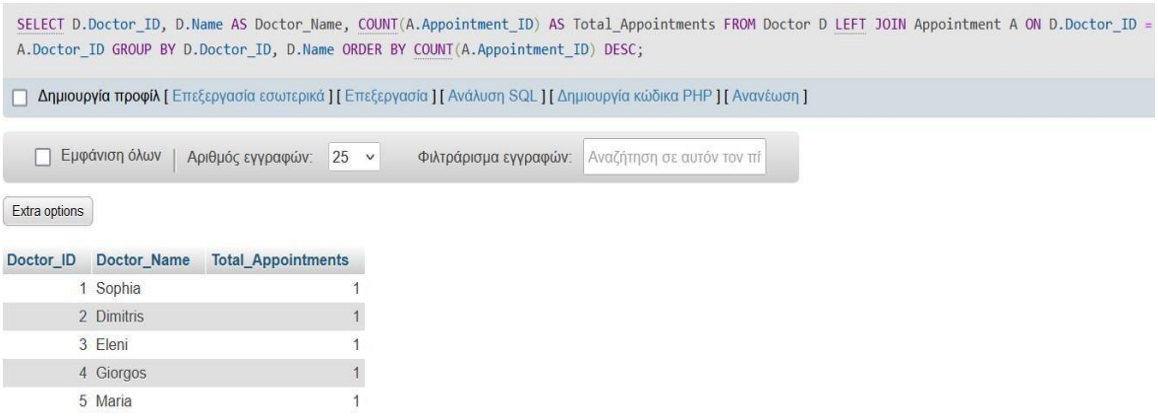
LEFT JOIN Appointment A ON D.Doctor_ID = A.Doctor_ID

GROUP BY D.Doctor_ID, D.Name

ORDER BY COUNT(A.Appointment_ID) DESC;
```

Επεξήγηση σε φυσική γλώσσα: Υπολογίζει πόσα ραντεβού έχει πραγματοποιήσει κάθε γιατρός και τα εμφανίζει ταξινομημένα από τον γιατρό με τα περισσότερα ραντεβού προς τα λιγότερα.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



Δύο ερωτήματα με «GROUP BY», εκ των οποίων το ένα να περιλαμβάνει και τη χρήση του «HAVING».

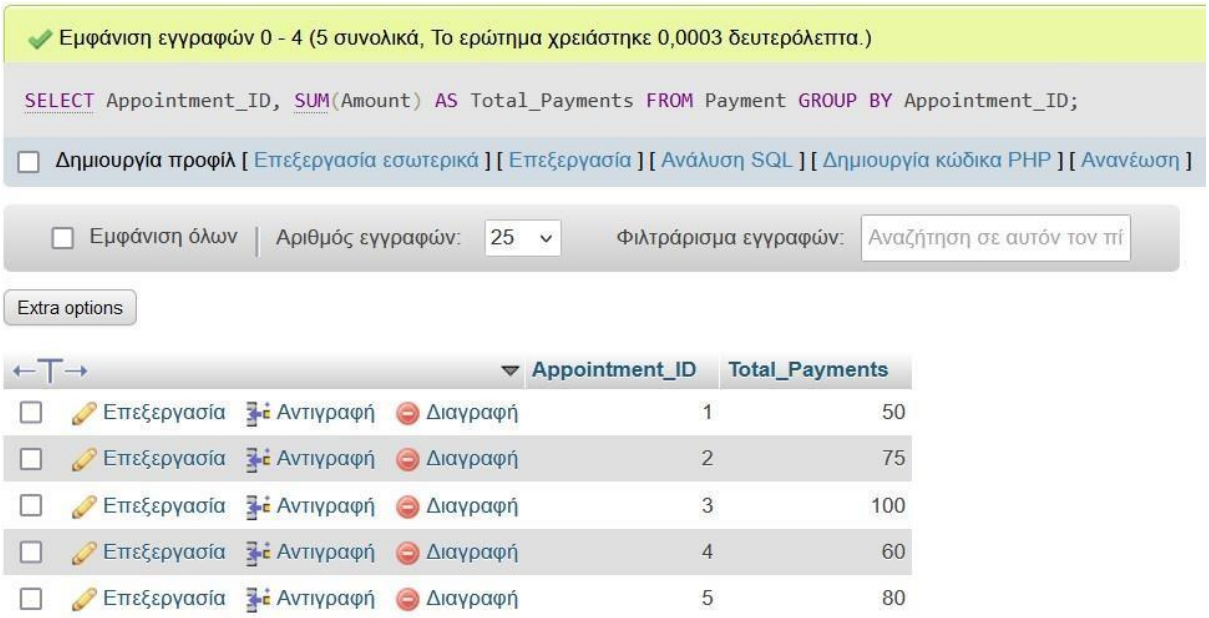
1ο ερώτημα

Υπολόγισε το συνολικό ποσό πληρωμών που έχουν γίνει ανά ραντεβού.

```
SELECT Appointment_ID, SUM(Amount) AS Total_Payments
FROM Payment
GROUP BY Appointment_ID;
```

Επεξήγηση σε φυσική γλώσσα: Υπολογίζει το συνολικό ποσό χρημάτων που έχει πληρωθεί για κάθε ραντεβού, ομαδοποιώντας τις πληρωμές με βάση το μοναδικό αναγνωριστικό του ραντεβού.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



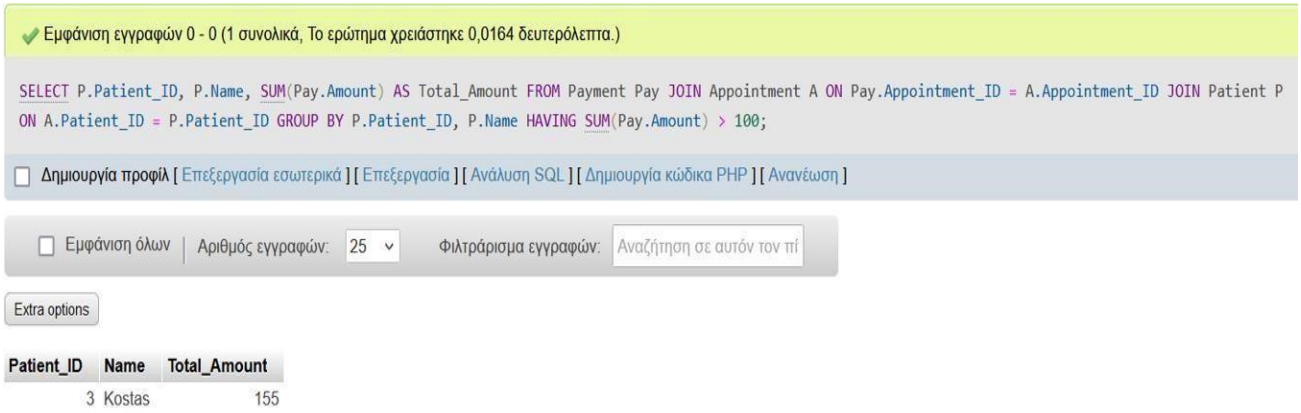
2ο ερώτημα

Υπολογίστε τους ασθενείς που έχουν συνολικά πληρώσει πάνω από 100€.

```
SELECT P.Patient_ID, P.Name, SUM(Pay.Amount) AS Total_Amount
FROM Payment Pay
JOIN Appointment A ON Pay.Appointment_ID = A.Appointment_ID
JOIN Patient P ON A.Patient_ID = P.Patient_ID
GROUP BY P.Patient_ID, P.Name
HAVING SUM(Pay.Amount) > 100;
```

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει τους ασθενείς που έχουν πληρώσει συνολικά ποσά άνω των 100€ για τα ραντεβού τους, μαζί με το όνομά τους και το συνολικό ποσό.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



Δύο ερωτήματα με «ORDER BY», όπου το ένα να ταξινομεί με βάση πολλαπλά κριτήρια.

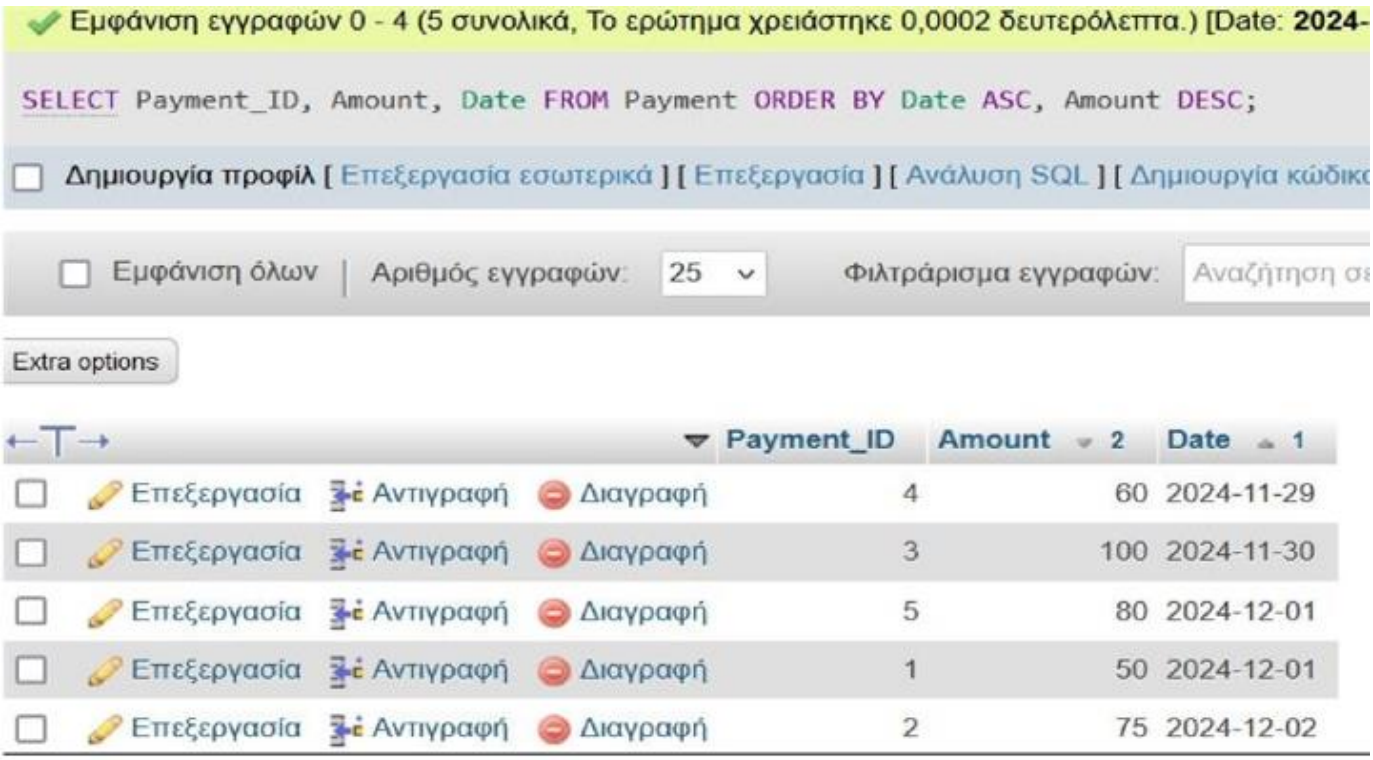
1ο ερώτημα

Υπολογίστε τις πληρώμες ταξινομημένες κατά ημερομηνία (αύξουσα σειρά) και στη συνέχεια κατά ποσό (φθίνουσα σειρά).

```
SELECT Payment_ID, Amount, Date
FROM Payment
ORDER BY Date ASC, Amount DESC;
```

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει όλες τις πληρωμές, ταξινομημένες πρώτα κατά

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



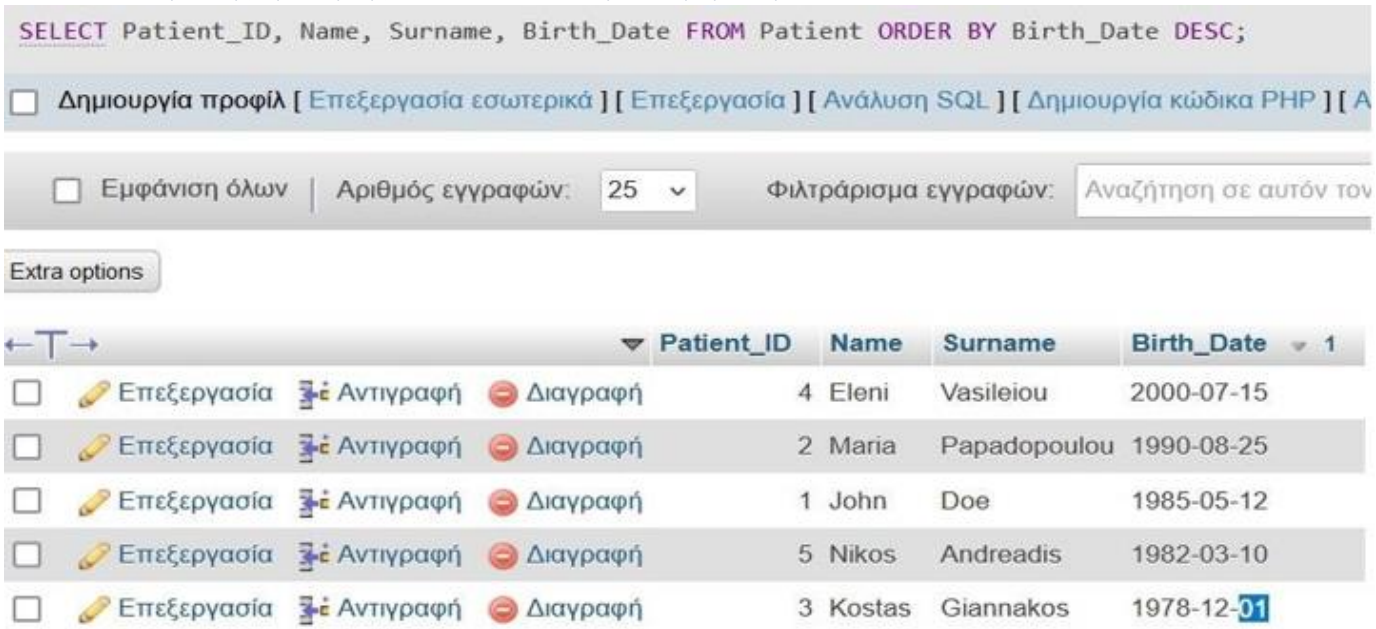
2ο ερώτημα

Υπολογίστε τα ονόματα των ασθενών , ταξινομημένα κατά ημερομηνία γέννησης από τους νεότερους προς τους μεγαλύτερους

```
SELECT Patient_ID, Name, Surname, Birth_Date
FROM Patient
ORDER BY Birth_Date DESC;
```

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει όλους τους ασθενείς, ταξινομημένους από τον νεότερο προς τον μεγαλύτερο, με βάση την ημερομηνία γέννησής τους.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



3 ερωτήματα που να περιλαμβάνουν διαφορετικούς τύπους JOIN (π.χ. INNER JOIN, LEFT JOIN, RIGHT JOIN), με τις εξής προδιαγραφές:

α) Ένα ερώτημα να συνδυάζει τουλάχιστον 3 πίνακες.

β) Ένα άλλο ερώτημα να συνδυάζει τουλάχιστον 4 πίνακες.

γ) Το τρίτο ερώτημα να χρησιμοποιεί διαφορετικό τύπο JOIN από τα άλλα δύο

1ο ερώτημα

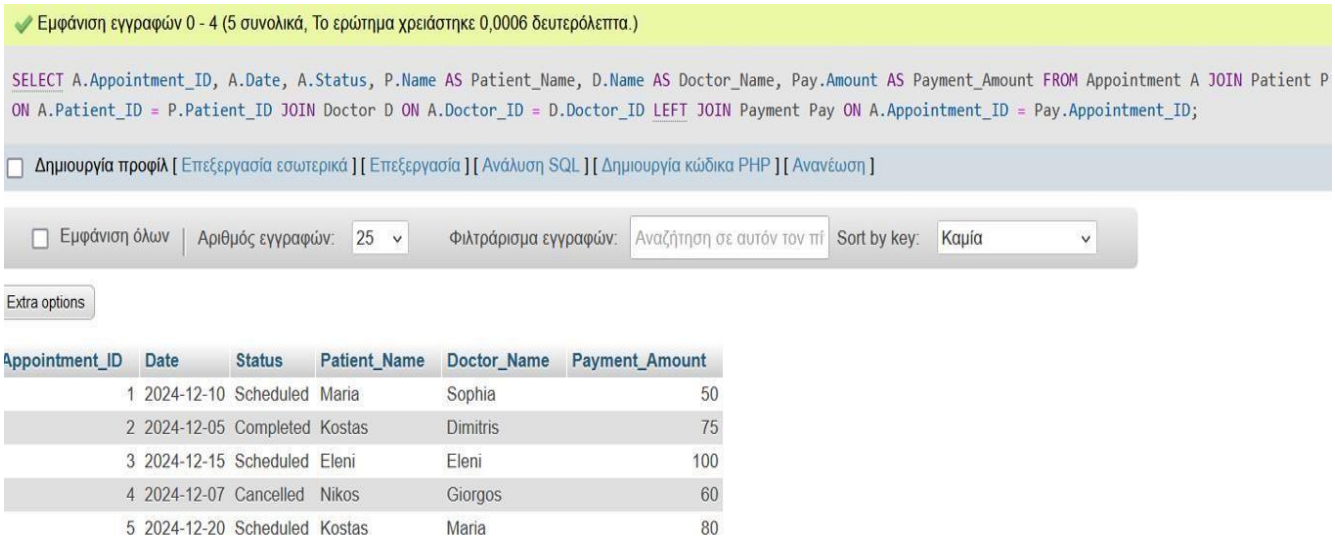
Υπολογίστε τις πληροφορίες για τα ραντεβού , τον γιατρό , τον ασθενή και το ποσό πληρωμής

```
SELECT
    A.Appointment_ID, A.Date, A.Status,
    P.Name AS Patient_Name,
    D.Name AS Doctor_Name,
    Pay.Amount AS Payment_Amount
FROM Appointment A
JOIN Patient P ON A.Patient_ID = P.Patient_ID
JOIN Doctor D ON A.Doctor_ID = D.Doctor_ID
```

LEFT JOIN Payment Pay ON A.Appointment_ID = Pay.Appointment_ID;

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει τα στοιχεία του ασθενούς και του ραντεβού, καθώς και το συνολικό ποσό πληρωμών για το συγκεκριμένο ραντεβού.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση


Εμφάνιση εγγραφών 0 - 4 (5 συνολικά, Το ερώτημα χρειάστηκε 0,0006 δευτερόλεπτα.)

```
SELECT A.Appointment_ID, A.Date, A.Status, P.Name AS Patient_Name, D.Name AS Doctor_Name, Pay.Amount AS Payment_Amount FROM Appointment A JOIN Patient P
ON A.Patient_ID = P.Patient_ID JOIN Doctor D ON A.Doctor_ID = D.Doctor_ID LEFT JOIN Payment Pay ON A.Appointment_ID = Pay.Appointment_ID;
```

Δημιουργία προφίλ [Επεξεργασία εσωτερικά] [Επεξεργασία] [Ανάλυση SQL] [Δημιουργία κώδικα PHP] [Ανανέωση]

Εμφάνιση όλων | Αριθμός εγγραφών: 25 | Φιλτράρισμα εγγραφών: Αναζήτηση σε αυτόν τον πί | Sort by key: Καμία

Extra options

Appointment_ID	Date	Status	Patient_Name	Doctor_Name	Payment_Amount
1	2024-12-10	Scheduled	Maria	Sophia	50
2	2024-12-05	Completed	Kostas	Dimitris	75
3	2024-12-15	Scheduled	Eleni	Eleni	100
4	2024-12-07	Cancelled	Nikos	Giorgos	60
5	2024-12-20	Scheduled	Kostas	Maria	80

2ο ερώτημα

Υπολογίστε τα στοιχεία του ραντεβού , το όνομα του γιατρού και του ασθενούς και τα φάρμακα που συνταγοφραφήθηκαν

```
SELECT
    A.Appointment_ID, A.Date AS Appointment_Date,
    D.Name AS Doctor_Name, D.Surname AS Doctor_Surname,
    P.Name AS Patient_Name, P.Surname AS Patient_Surname,
    PR.Medicine_Name, PR.Dosage
FROM Appointment A
JOIN Doctor D ON A.Doctor_ID = D.Doctor_ID
JOIN Patient P ON A.Patient_ID = P.Patient_ID
JOIN Prescription PR ON A.Appointment_ID = PR.Appointment_ID;
```

Επεξήγηση σε φυσική γλώσσα: Εμφάνισε τα στοιχεία κάθε ραντεβού, μαζί Το μοναδικό ID και την ημερομηνία του ραντεβού. Το όνομα και το επώνυμο του γιατρού που συμμετείχε στο ραντεβού.Το όνομα και το επώνυμο του ασθενούς που παρακολούθησε το ραντεβού. Τα φάρμακα που συνταγογραφήθηκαν κατά το ραντεβού, μαζί με τη δοσολογία τους.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση

```
SELECT A.Appointment_ID, A.Date AS Appointment_Date, D.Name AS Doctor_Name, D.Surname AS Doctor_Surname, P.Name AS Patient_Name, P.Surname AS
Patient_Surname, PR.Medicine_Name, PR.Dosage FROM Appointment A JOIN Doctor D ON A.Doctor_ID = D.Doctor_ID JOIN Patient P ON A.Patient_ID = P.Patient_ID
JOIN Prescription PR ON A.Appointment_ID = PR.Appointment_ID;
```

Δημιουργία προφίλ [Επεξεργασία εσωτερικά] [Επεξεργασία] [Ανάλυση SQL] [Δημιουργία κώδικα PHP] [Ανανέωση]

Εμφάνιση όλων | Αριθμός εγγραφών: 25 | Φιλτράρισμα εγγραφών: Αναζήτηση σε αυτόν τον πί

Extra options

Appointment_ID	Appointment_Date	Doctor_Name	Doctor_Surname	Patient_Name	Patient_Surname	Medicine_Name	Dosage
1	2024-12-10	Sophia	Papadopoulou	Maria	Papadopoulou	Amoxicillin	500mg
3	2024-12-15	Eleni	Andreadou	Eleni	Vasileiou	Loratadine	10mg
4	2024-12-07	Giorgos	Ioannou	Nikos	Andreadis	Ibuprofen	200mg
2	2024-12-05	Dimitris	Nikolaidis	Kostas	Giannakos	Amlodipine	5mg
1	2024-12-10	Sophia	Papadopoulou	Maria	Papadopoulou	Paracetamol	250mg

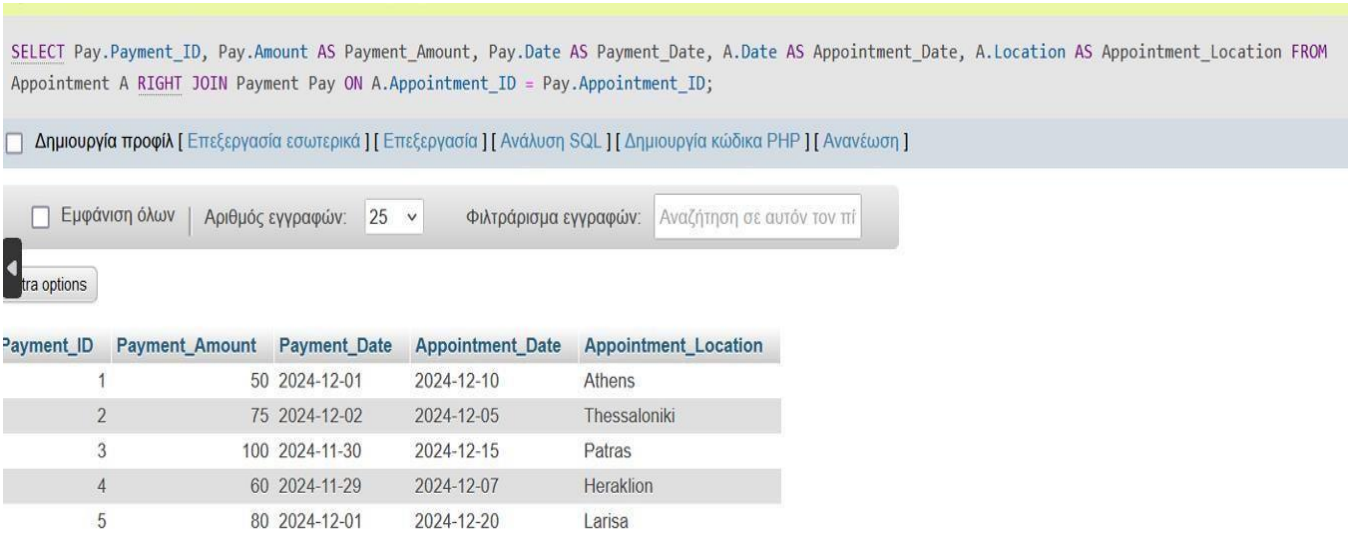
3ο ερώτημα

Υπολογίστε όλες τις πληρωμές και αν υπάρχουν τα στοιχεία του ραντεβού που σχετίζονται με αυτές

```
SELECT
    Pay.Payment_ID, Pay.Amount AS Payment_Amount, Pay.Date AS Payment_Date,
    A.Date AS Appointment_Date, A.Location AS Appointment_Location
FROM Appointment A
RIGHT JOIN Payment Pay ON A.Appointment_ID = Pay.Appointment_ID;
```

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει όλες τις πληρωμές (ID, ποσό και ημερομηνία πληρωμής) και, αν υπάρχουν, την ημερομηνία και την τοποθεσία του σχετικού ραντεβού. Αν μια πληρωμή δεν συνδέεται με ραντεβού, εμφανίζεται μόνο η πληρωμή.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



```
SELECT Pay.Payment_ID, Pay.Amount AS Payment_Amount, Pay.Date AS Payment_Date, A.Date AS Appointment_Date, A.Location AS Appointment_Location FROM
Appointment A RIGHT JOIN Payment Pay ON A.Appointment_ID = Pay.Appointment_ID;
```

Δημιουργία προφίλ [Επεξεργασία εσωτερικά] [Επεξεργασία] [Ανάλυση SQL] [Δημιουργία κώδικα PHP] [Ανανέωση]

Εμφάνιση όλων | Αριθμός εγγραφών: 25 Φιλτράρισμα εγγραφών: Αναζήτηση σε αυτόν τον πί

tra options

Payment_ID	Payment_Amount	Payment_Date	Appointment_Date	Appointment_Location
1	50	2024-12-01	2024-12-10	Athens
2	75	2024-12-02	2024-12-05	Thessaloniki
3	100	2024-11-30	2024-12-15	Patras
4	60	2024-11-29	2024-12-07	Heraklion
5	80	2024-12-01	2024-12-20	Larisa

2 ερωτήματα που να περιλαμβάνουν ένα εμφωλευμένο SELECT (subquery), με τουλάχιστον ένα να χρησιμοποιεί τη subquery στη ρήτρα WHERE.

1ο ερώτημα

Υπολογίστε τα ονόματα των ασθενών που έχουν υποβληθεί σε κάποια εξέταση

```
SELECT Name
FROM Patient
WHERE Patient_ID IN (
   SELECT DISTINCT Patient_ID
   FROM Medical_Test  );
```

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει τα ονόματα των ασθενών που έχουν συμμετάσχει σε εξετάσεις, με βάση τα δεδομένα από τις εγγραφές εξετάσεων.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση



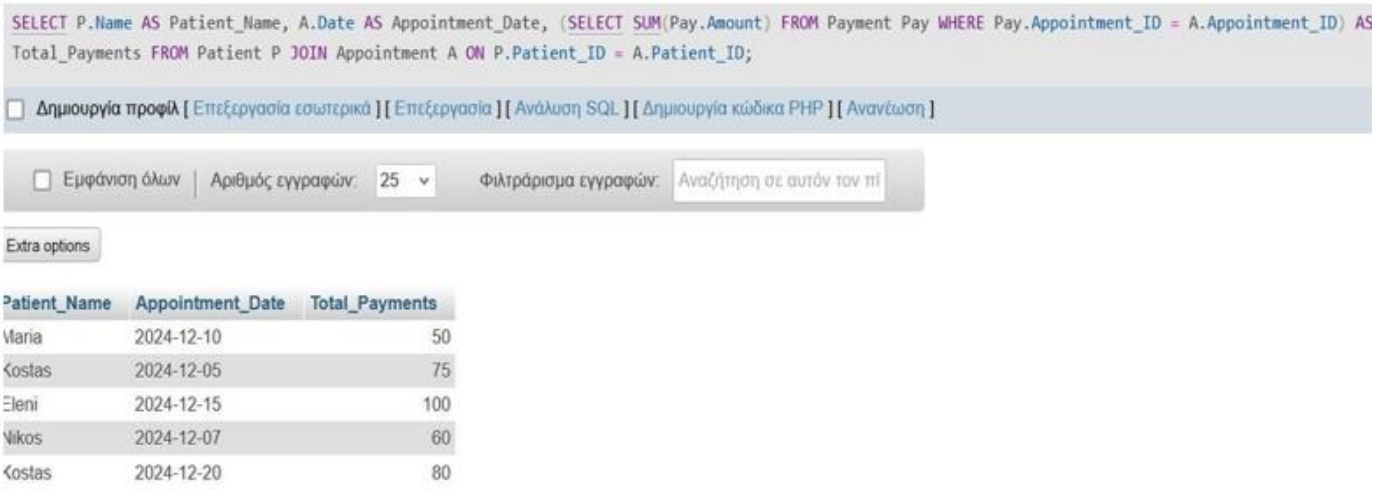
2ο ερώτημα

Υπολογίστε τα στοιχεία των ασθενών και ραντεβού μαζί με το συνολικό ποσό πληρωμών

```
SELECT
P.Name AS Patient_Name,
A.Date AS Appointment_Date,
   (SELECT SUM(Pay.Amount)
    FROM Payment Pay
    WHERE Pay.Appointment_ID = A.Appointment_ID) AS Total_Payments
FROM Patient P
JOIN Appointment A ON P.Patient_ID = A.Patient_ID
```

Επεξήγηση σε φυσική γλώσσα: Εμφανίζει το όνομα κάθε ασθενούς, την ημερομηνία του ραντεβού του και το συνολικό ποσό πληρωμών που έχει πραγματοποιηθεί για το συγκεκριμένο ραντεβού.

Το αποτέλεσμα της ερώτησης όταν το εκτελούμε στην βάση

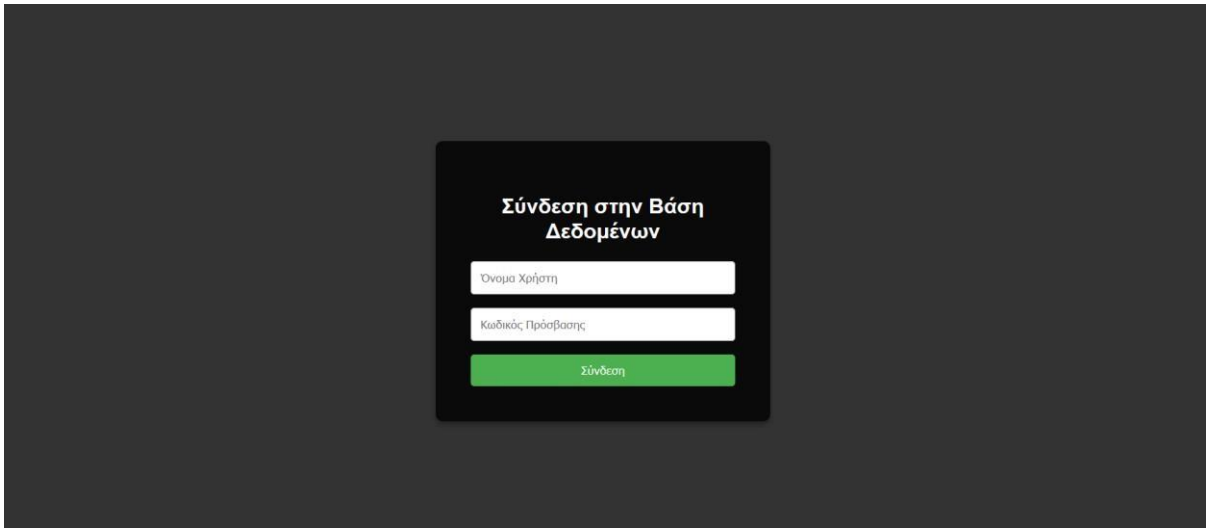


9. Υλοποίηση Εφαρμογής

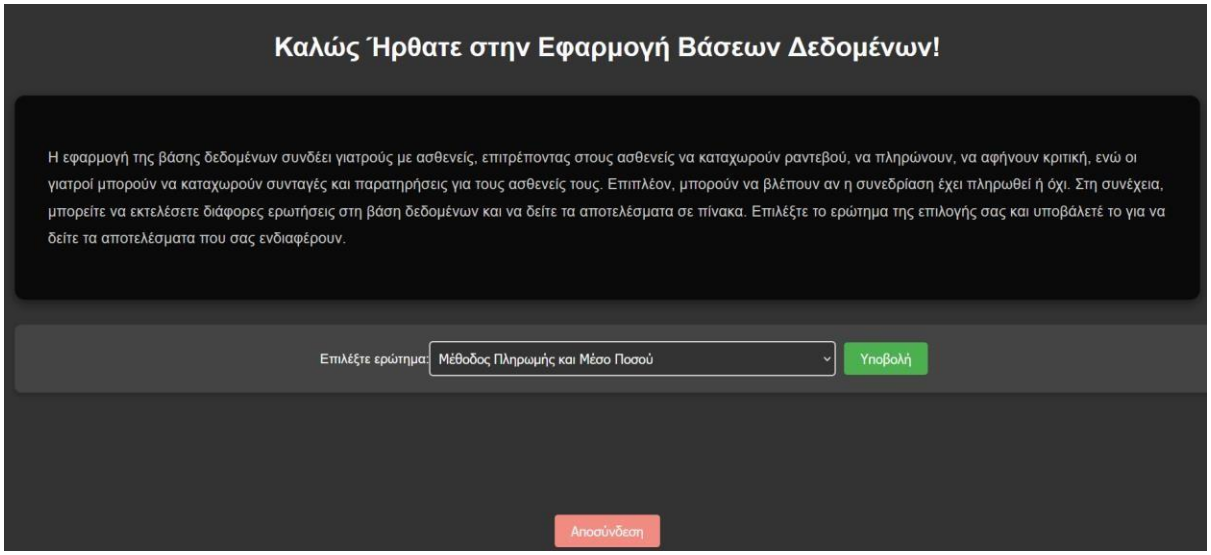
Σκοπός της εφαρμογής είναι υλοποίηση μιας διεπαφής που θα συνδέεται με την βάση δεδομένων που. Ο χρήστης θα μπορεί να επιλέξει κάποια από τις ερωτήσεις SQL και θα μπορεί να τις εκτελέσει μέσω ενός μενού που θα εμφανίζεται και τα αποτελέσματα του κάθε ερωτήματος θα εμφανίζονται σε κατάλληλο πίνακα.

Αρχικά ο χρήστης πρέπει να εισάγει τα στοιχεία του (username και password) για να μπορεί να πραγματοποιήσει είσοδο στην εφαρμογή της βάσης δεδομένων. Αξίζει να σημειωθεί πως για κάθε χρήστη-επισκέπτη έχουμε δημιουργήσει έναν πίνακα στην βάση δεδομένων μας με όνομα users οπού εκεί αποθηκεύονται τα στοιχεία σύνδεσης του. (Δημιουργούμε χρήστες που αποθηκεύονται στην βάση με την βοήθεια της PHP έτσι γίνονται οι εγγραφές στον πίνακα) . Όμως πρέπει να σημειώσουμε πως ο κωδικός που αποθηκεύεται είναι κρυπτογραφημένος. Εφαρμόζουμε αυτήν την διαδικασία για την μεγιστοποίηση της ασφάλειας και να μειώσουμε την πιθανότητα παραβίασης της. Ακόμη στα ερωτήματα που ζητείτε από τον χρήστη να εισάγει δεδομένα για την εκτέλεση ενός ερωτήματος χρησιμοποιούμε prepared statements. Βασικός σκοπός της χρήσης του είναι η αποτροπή του **SQL Injection** .Το **SQL Injection** είναι μία από τις πιο κοινές επιθέσεις σε εφαρμογές με βάση δεδομένων. Ένας εισβολέας μπορεί να εισάγει κακόβουλο κώδικα στις εισόδους της εφαρμογής σας, που στη συνέχεια εκτελείται στη βάση δεδομένων. Τα **Prepared Statements** προλαμβάνουν αυτό το πρόβλημα γιατί διαχωρίζουν τα δεδομένα από τον κώδικα SQL. Οι παράμετροι εισόδου θεωρούνται ως δεδομένα και **δεν εκτελούνται ως μέρος της εντολής SQL**.

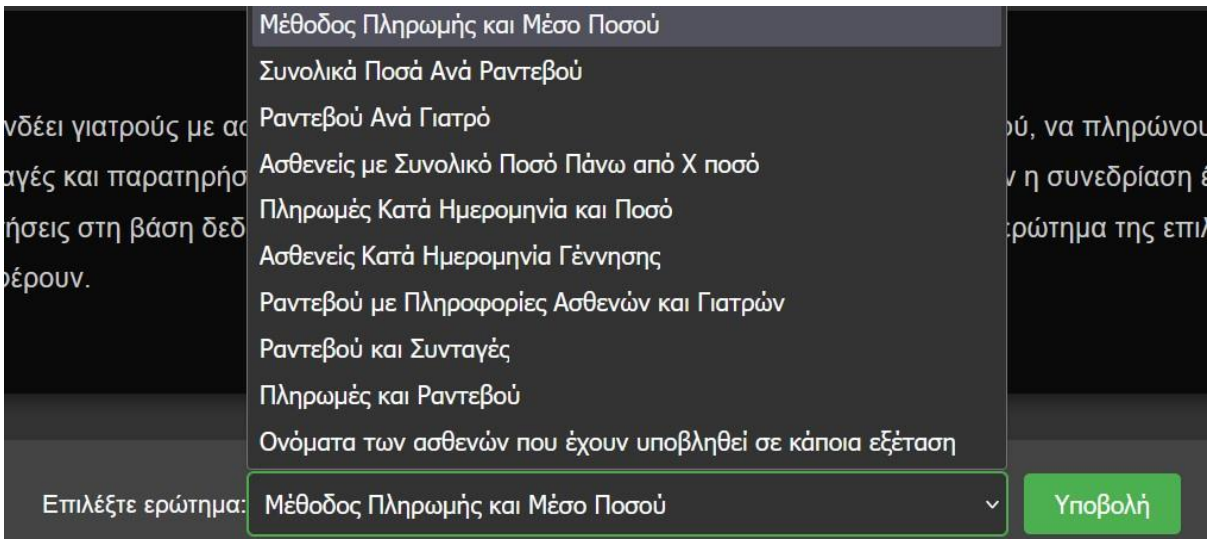
Ξεκινάμε με την είσοδο του χρήστη



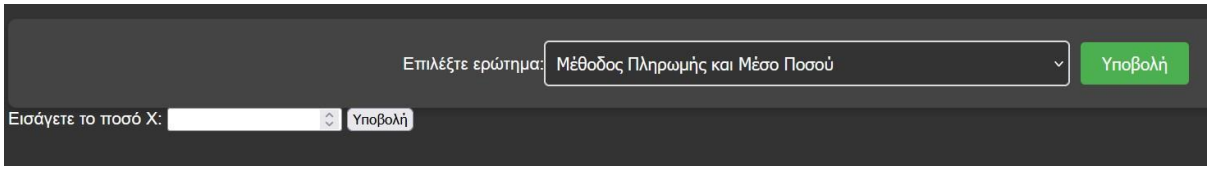
Ο χρήστης εισάγει τα στοιχεία του και κάνει είσοδο στην βάση.



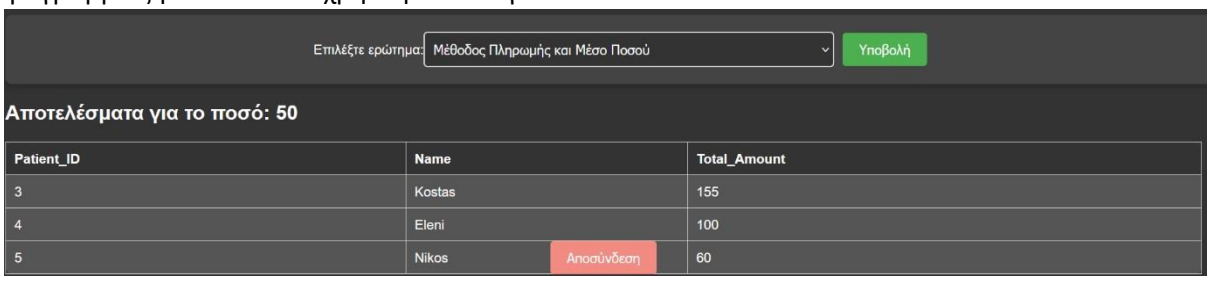
Εδώ βλέπουμε το κύριο μενού της εφαρμογής. Ένα κείμενο που περιγραφεί την λειτουργεία και τον σκοπό της εφαρμογής. Παρακάτω βρίσκεται η δυνατότητα επιλογής του ερωτήματος που εμφανίζεται σε ένα μενού που θα δούμε παρακάτω. Θα παρατηρήσουμε ότι κεντρικά χαμηλά βρίσκεται το κουμπί της αποσύνδεσής όπου μπορεί ο χρήστης να κάνει έξοδο από την βάση.



Το μενού ερωτήσεων που μπορεί να επιλέξει ο χρήστης.



Επιλέγουμε το ερώτημα (Ασθενείς με συνολικό ποσό πάνω από Χ ποσό). Παρατηρούμε πως το πρόγραμμα ζητάει από τον χρήστη να εισάγει το ποσό Χ.



Ο χρήστης εισάγει το 50 και το πρόγραμμα εμφανίζει τον αντίστοιχο πίνακα έπειτα από την εισαγωγή ποσού από τον χρήστη.

Τέλος ο χρήστης κάνει αποσύνδεση και κάνει έξοδο από το πρόγραμμα

10. Υλοποίηση Βάσης Γραφοδεδομένων

Τι είναι η βάση γραφοδεδομένων και γιατί την χρησιμοποιούμε ;

Τα έργα συχνά ασχολούνται με μεγάλες ποσότητες πολύπλοκων δεδομένων και οι βάσεις δεδομένων γράφων μπορούν να αποτελέσουν ένα ισχυρό εργαλείο. Υπάρχουν και άλλοι τρόποι αποθήκευσης δεδομένων ως αντικείμενα και συνδέσεις, όπως για παράδειγμα οι σχεσιακές βάσεις δεδομένων. Ωστόσο, οι σχεσιακές βάσεις δεδομένων χρησιμοποιούν υπολογιστικά δαπανηρές πράξεις JOIN ή διασταυρώσεις, οι οποίες συχνά συνδέονται με ένα άκαμπτο μοντέλο δεδομένων. Οι βάσεις δεδομένων γράφων δεν χρησιμοποιούν JOINs. Αντίθετα, οι σχέσεις αποθηκεύονται εγγενώς μαζί με τα στοιχεία δεδομένων (κόμβους) σε μια πιο ευέλικτη μορφή, η οποία επιτρέπει τη βελτιστοποίηση της διάσχισης των δεδομένων και την πρόσβαση σε εκατομμύρια συνδέσεις ανά δευτερόλεπτο.

Πως λειτουργεί ;

Μια **βάση γραφοδεδομένων** είναι ένας τύπος βάσης δεδομένων που χρησιμοποιεί **κόμβους (nodes)** και **σχέσεις (relationships)** για να αποθηκεύσει, να διαχειριστεί και να αναλύσει δεδομένα. Σε αντίθεση με τις παραδοσιακές **σχεσιακές βάσεις δεδομένων (SQL)**, οι βάσεις γραφοδεδομένων επικεντρώνονται στις **συνδέσεις** μεταξύ των δεδομένων και τις χειρίζονται πιο αποδοτικά.

Οι κόμβοι είναι οντότητες στο γράφο που μπορούν να:

- Να επισημανθούν με ετικέτες που αντιπροσωπεύουν τους διάφορους ρόλους τους σε έναν τομέα (π.χ., πρόσωπο).
- Να κατέχουν οποιονδήποτε αριθμό ζευγών κλειδιών-τιμών ως ιδιότητες (π.χ. όνομα).
- Να δεικτοδοτούνται και να δεσμεύονται από περιορισμούς.

Οι σχέσεις παρέχουν ονομαστικές συνδέσεις (Relationships) μεταξύ δύο κόμβων (π.χ., Person - LOVES - Person) και αυτές:

- Πρέπει πάντα να έχουν έναν κόμβο αρχής, έναν κόμβο τέλους και ακριβώς έναν τύπο.
- Πρέπει να έχουν κατεύθυνση.
- Μπορούν να έχουν ιδιότητες, όπως οι κόμβοι.
- Οι κόμβοι μπορούν να έχουν πολλαπλές σχέσεις διαφόρων τύπων χωρίς να θυσιάζεται η απόδοση.

Συνοψίζοντας, οι κόμβοι και οι σχέσεις είναι ένας αποτελεσματικός και ευέλικτος τρόπος αποθήκευσης δεδομένων, καθώς σας επιτρέπουν να:

- Να δημιουργείτε διελεύσεις σε μεγάλους γράφους τόσο για βάθος όσο και για πλάτος.
- Να επεκτείνετε τη βάση δεδομένων σας σε δισεκατομμύρια κόμβους.
- Να σχεδιάζετε ευέλικτα μοντέλα δεδομένων γράφων ιδιοτήτων που μπορούν να προσαρμόζονται με την πάροδο του χρόνου.

Βήματα μετατροπής της σχεσιακής βάσης σε βάση γραφοδεδομένων

Η μετατροπή μιας σχεσιακής βάσης δεδομένων (RDBMS) σε μια βάση δεδομένων γραφημάτων (graph database) είναι μια διαδικασία που απαιτεί καλή κατανόηση της δομής των δεδομένων και των σχέσεων.

Βήματα μετατροπής

1.Κατανόηση πινάκων

Αρχικά πρέπει να κατανοήσουμε τους πίνακες που έχουμε στην σχεσιακή βάση δεδομένων. Συνήθως κάθε πίνακας αντιπροσωπεύει μια οντότητα.

2. Ανάλυση των σχέσεων

Πρέπει να αναλύσουμε τους τύπους σχέσεων μεταξύ των πινάκων δηλαδή ποιοι πίνακες σχετίζονται μεταξύ τους με τις σχέσεις (One-to-many, Many-to-many, κλπ.)

3. Σχεδίαση οντοτήτων και σχέσεων

Στη βάση γραφημάτων οι πίνακες από την σχεσιακή βάση θα μετατραπούν σε κόμβους. Ενώ ακόμη οι σχέσεις μεταξύ των πινάκων θα μετατραπούν σε Relationships. Αυτές οι σχέσεις στην βάση γράφων θα συνδέουν του κόμβους και θα καθορίζουν τις αλληλεπιδράσεις μεταξύ τους.

4. Ιδιότητες

Κάθε κόμβος ή σχέση μπορεί να έχει ιδιότητες οι οποίες αντιστοιχούν στα πεδία των πινάκων της σχεσιακής βάσης.

Επομένως στην δικιά μας περίπτωση θα εφαρμόσουμε τα βήματα που αναφέραμε παραπάνω:

1. Κατανόηση πινάκων

Έχουμε τους πίνακες (οντότητες):

Doctor

Patient

Appointment

Medical_History

Review

Payment

Prescription

Medical_test

Condition

2. Ανάλυση των σχέσεων

Αναλύουμε βλέποντας το διάγραμμα τις σχέσεις των πινάκων παραδείγματος χάρη:

[**Doctor -> Appointment** (One-to- One_or_Many)]

[**Patient -> Medical_History** (One-to-One)]

[**Doctor -> Medical_test** (One-to-Zero_or_Many)]

[**Patient -> Appointment** (One-to-One_or_Many)]

3. Σχεδίαση οντοτήτων και σχέσεων

Οντότητες

Ο πίνακας Doctor θα μετατραπεί σε **κόμβο** τύπου Doctor.

Ο πίνακας Patient θα μετατραπεί σε **κόμβο** τύπου Patient.

Ο πίνακας Appointment θα μετατραπεί σε **κόμβο** τύπου Appointment

Ο πίνακας Medical_history θα μετατραπεί σε **κόμβο** τύπου Medical_history

Η ίδια διαδικασία και με τις άλλες οντότητες.

Σχέσεις (Relationships)

Η σχέση μεταξύ Doctor και Appointment θα αναπαρίσταται με μια σχέση τύπου **HAS_APPOINTMENT.**

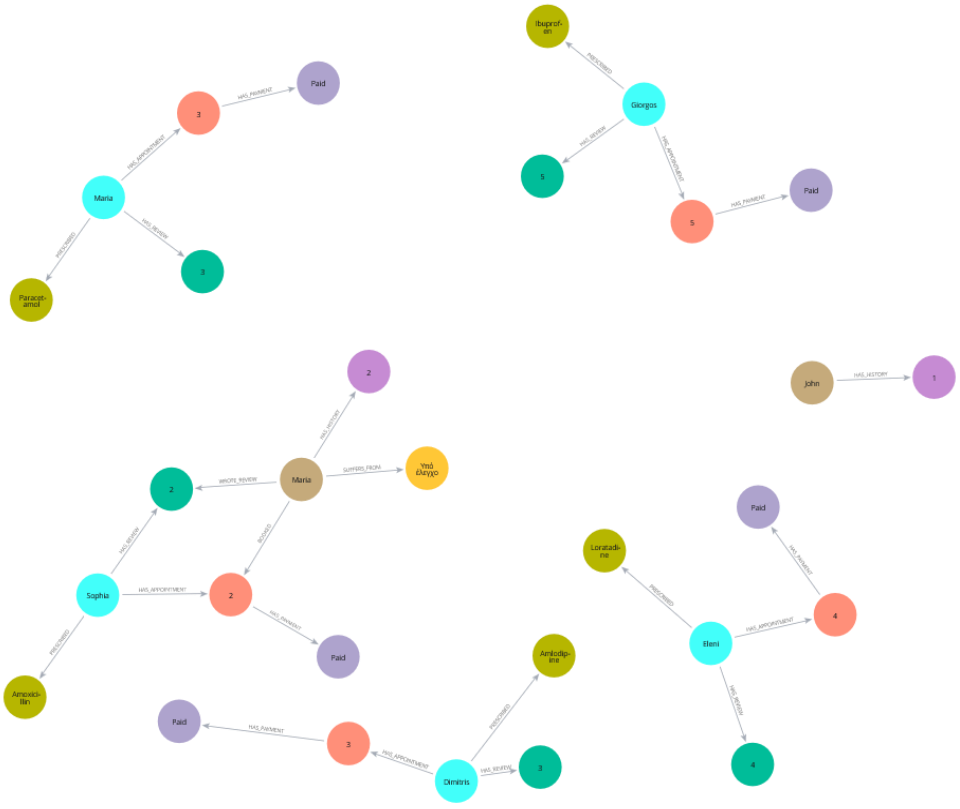
Η σχέση μεταξύ Patient και Appointment θα να αναπαρίσταται με μια σχέση τύπου **BOOKED.**

Η σχέση μεταξύ Patient και Medical_history θα αναπαρίσταται με μια σχέση τύπου **HAS_HISTORY**

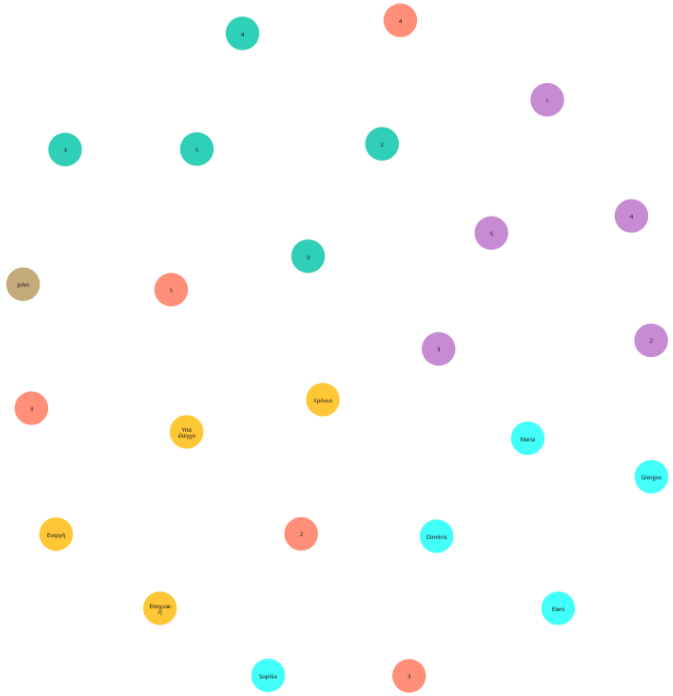
Η ίδια διαδικασία και με τις άλλες σχέσεις.

11. Το μοντέλο γραφοδεδομένων

Διάγραμμα με τις σχέσεις (Relationships)



Διάγραμμα οντοτήτων



Σημείωση: Τα διαγράμματα και η υλοποίηση της βάσης γράφων έγιναν με την χρήση του Neo4j aura

12. Ο CYPHER κώδικας για την δημιουργία της βάσης

Η οντότητα Appointment

```
CREATE

 (:Appointment {Appointment_ID: 1, Status: 'Scheduled', Notes: 'Follow-up visit', Date: '2024-12-10', Location: 'Athens', Price: 50, Time: '10:00', Type: 'Consultation', Patient_ID: 2, Doctor_ID: 1}),

 (:Appointment {Appointment_ID: 2, Status: 'Completed', Notes: 'First visit', Date: '2024-12-05', Location: 'Thessaloniki', Price: 70, Time: '12:30', Type: 'Checkup', Patient_ID: 3, Doctor_ID: 2}),

 (:Appointment {Appointment_ID: 3, Status: 'Scheduled', Notes: 'Routine check', Date: '2024-12-15', Location: 'Patras', Price: 30, Time: '14:00', Type: 'Consultation', Patient_ID: 4, Doctor_ID: 3}),

 (:Appointment {Appointment_ID: 4, Status: 'Cancelled', Notes: 'Patient requested cancellation', Date: '2024-12-07', Location: 'Heraklion', Price: 40, Time: '09:00', Type: 'Follow-up', Patient_ID: 5, Doctor_ID: 4}),

 (:Appointment {Appointment_ID: 5, Status: 'Scheduled', Notes: 'MRI referral', Date: '2024-12-20', Location: 'Larisa', Price: 100, Time: '11:15', Type: 'Specialist', Patient_ID: 3, Doctor_ID: 5});
```

Η οντότητα Condition

```
CREATE

 (:Condition {Condition_ID: 1, Time: '2024-12-01', Status: 'Υπό έλεγχο', Disease_Type: 'Υπέρταση', History_ID: 1}),

 (:Condition {Condition_ID: 2, Time: '2024-11-15', Status: 'Ενεργή', Disease_Type: 'Διαβήτης τύπου 2', History_ID: 2}),

 (:Condition {Condition_ID: 3, Time: '2024-04-20', Status: 'Εποχιακή', Disease_Type: 'Αλλεργική Ρινίτιδα', History_ID: 3}),

 (:Condition {Condition_ID: 4, Time: '2024-03-10', Status: 'Χρόνια', Disease_Type: 'Γαστροοισοφαγική Παλινδρόμηση', History_ID: 4});
```

Η οντότητα Doctor

```
CREATE

 (:Doctor {Doctor_ID: 1, Name: 'Sophia', Surname: 'Papadopoulou', Specialty: 'Cardiologist', Birth_Date: '1980-04-15', Phone: 6941234567, Email: 'sophia.pap@example.com', City: 'Athens', Street: 'Vasilisis Sofias 10', Postal_Code: 11521}),

 (:Doctor {Doctor_ID: 2, Name: 'Dimitris', Surname: 'Nikolaidis', Specialty: 'Orthopedic', Birth_Date: '1975-11-20', Phone: 6934567890, Email: 'dimitris.niko@example.com', City: 'Thessaloniki', Street: 'Tsimiski 45', Postal_Code: 54623}),

 (:Doctor {Doctor_ID: 3, Name: 'Eleni', Surname: 'Andreadou', Specialty: 'Pediatrician', Birth_Date: '1988-09-12', Phone: 6979876543, Email: 'eleni.and@example.com', City: 'Patras', Street: 'Korinthou 15', Postal_Code: 26221}),

 (:Doctor {Doctor_ID: 4, Name: 'Giorgos', Surname: 'Ioannou', Specialty: 'Dermatologist', Birth_Date: '1985-06-05', Phone: 6981234789, Email: 'giorgos.io@example.com', City: 'Heraklion', Street: 'Lydias 32', Postal_Code: 71202}),

 (:Doctor {Doctor_ID: 5, Name: 'Maria', Surname: 'Vasileiou', Specialty: 'Neurologist', Birth_Date: '1990-01-18', Phone: 6951234987, Email: 'maria.vas@example.com', City: 'Larisa', Street: 'Kountouriotou 78', Postal_Code: 41222});
```

Η οντότητα Medical_History

CREATE

(:MedicalHistory {History_ID: 1, Medicines: 'Paracetamol, Ibuprofen', Notes: 'Υψηλή αρτηριακή πίεση, παρακολούθηση.', Allergies: 'Πενικιλίνη', Patient_ID: 2}),

(:MedicalHistory {History_ID: 2, Medicines: 'Metformin', Notes: 'Διαβήτης τύπου 2, τακτική λήψη φαρμάκου.', Allergies: 'Καμία γνωστή αλλεργία', Patient_ID: 3}),

(:MedicalHistory {History_ID: 3, Medicines: 'Cetirizine', Notes: 'Εποχιακή αλλεργία, αγωγή κατά την άνοιξη.', Allergies: 'Γύρη', Patient_ID: 4}),

(:MedicalHistory {History_ID: 4, Medicines: 'Omeprazole', Notes: 'Γαστροοισοφαγική παλινδρόμηση, καθημερινή χρήση.', Allergies: 'Ασπιρίνη', Patient_ID: 5}),

(:MedicalHistory {History_ID: 5, Medicines: 'Amoxicillin', Notes: 'Θεραπεία πρόσφατης λοίμωξης, παρακολούθηση.', Allergies: 'Πενικιλίνη', Patient_ID: 1});

Η οντότητα Medical_test

CREATE

(:MedicalTest {Test_ID: 1, Test_Date: '2024-12-10', Test_Type: 'Blood Test', Notes: 'Routine blood test', Result: 'Normal', Doctor_ID: 1, Appointment_ID: 1, Patient_ID: 2}),

(:MedicalTest {Test_ID: 2, Test_Date: '2024-12-05', Test_Type: 'X-Ray', Notes: 'X-ray for back pain', Result: 'No issues detected', Doctor_ID: 2, Appointment_ID: 2, Patient_ID: 3}),

(:MedicalTest {Test_ID: 3, Test_Date: '2024-12-15', Test_Type: 'Allergy Test', Notes: 'Skin test for allergies', Result: 'Positive for pollen', Doctor_ID: 3, Appointment_ID: 3, Patient_ID: 4}),

(:MedicalTest {Test_ID: 4, Test_Date: '2024-12-07', Test_Type: 'MRI', Notes: 'MRI for knee pain', Result: 'Signs of mild arthritis', Doctor_ID: 4, Appointment_ID: 4, Patient_ID: 5}),

(:MedicalTest {Test_ID: 5, Test_Date: '2024-12-20', Test_Type: 'EEG', Notes: 'Neurological assessment', Result: 'Normal brain activity', Doctor_ID: 5, Appointment_ID: 5, Patient_ID: 3});

Η οντότητα Patient

CREATE

(:Patient {Patient_ID: 1, Name: 'John', Surname: 'Doe', Social_Sec_Num: 123456789, Birth_Date: '1985-05-12', Phone: 6971234567, Email: 'john.doe@example.com', City: 'Athens', Postal_Code: 10435, Street_Num: '15A'}),

(:Patient {Patient_ID: 2, Name: 'Maria', Surname: 'Papadopoulou', Social_Sec_Num: 987654321, Birth_Date: '1990-08-25', Phone: 6987654321, Email: 'maria.pap@example.com', City: 'Thessaloniki', Postal_Code: 54623, Street_Num: '25'}),

(:Patient {Patient_ID: 3, Name: 'Kostas', Surname: 'Giannakos', Social_Sec_Num: 456789123, Birth_Date: '1978-12-01', Phone: 6941237890, Email: 'kostas.giannakos@example.com', City: 'Patra', Postal_Code: 26223, Street_Num: '10'}),

(:Patient {Patient_ID: 4, Name: 'Eleni', Surname: 'Vasileiou', Social_Sec_Num: 789123456, Birth_Date: '2000-07-15', Phone: 6934567890, Email: 'eleni.vas@example.com', City: 'Heraklion', Postal_Code: 71202, Street_Num: '5'}),

(:Patient {Patient_ID: 5, Name: 'Nikos', Surname: 'Andreadis', Social_Sec_Num: 159753684, Birth_Date: '1982-03-10', Phone: 6901234567, Email: 'nikos.and@example.com', City: 'Larisa', Postal_Code: 41222, Street_Num: '20'});

Η οντότητα Payment

CREATE

(:Payment {Payment_ID: 1, Status: 'Paid', Date: '2024-12-01', Amount: 50, Payment_Method: 'Credit Card', Appointment_ID: 1}),

(:Payment {Payment_ID: 2, Status: 'Paid', Date: '2024-12-02', Amount: 75, Payment_Method: 'Cash', Appointment_ID: 2}),

(:Payment {Payment_ID: 3, Status: 'Paid', Date: '2024-11-30', Amount: 100, Payment_Method: 'Debit Card', Appointment_ID: 3}),

(:Payment {Payment_ID: 4, Status: 'Paid', Date: '2024-11-29', Amount: 60, Payment_Method: 'Bank Transfer', Appointment_ID: 4}),

(:Payment {Payment_ID: 5, Status: 'Paid', Date: '2024-12-01', Amount: 80, Payment_Method: 'Cash', Appointment_ID: 5});

Η οντότητα Prescription

CREATE

(:Prescription {Prescription_ID: 1, Frequency: 'Twice daily', Dosage: '500mg', Medicine_Name: 'Amoxicillin', Notes: 'For throat infection', Duration: 7, Patient_ID: 2, Appointment_ID: 1, Doctor_ID: 1, Test_ID: 1}),

(:Prescription {Prescription_ID: 2, Frequency: 'Once daily', Dosage: '10mg', Medicine_Name: 'Loratadine', Notes: 'For allergies', Duration: 14, Patient_ID: 4, Appointment_ID: 3, Doctor_ID: 3, Test_ID: 3}),

(:Prescription {Prescription_ID: 3, Frequency: 'Every 8 hours', Dosage: '200mg', Medicine_Name: 'Ibuprofen', Notes: 'For knee pain', Duration: 5, Patient_ID: 5, Appointment_ID: 4, Doctor_ID: 4, Test_ID: 4}),

(:Prescription {Prescription_ID: 4, Frequency: 'Once daily', Dosage: '5mg', Medicine_Name: 'Amlodipine', Notes: 'For blood pressure', Duration: 30, Patient_ID: 3, Appointment_ID: 2, Doctor_ID: 2, Test_ID: 2}),

(:Prescription {Prescription_ID: 5, Frequency: 'Three times daily', Dosage: '250mg', Medicine_Name: 'Paracetamol', Notes: 'For fever and headache', Duration: 3, Patient_ID: 4, Appointment_ID: 1, Doctor_ID: 5, Test_ID: 5});

Η οντότητα Review

CREATE

(:Review {Review_ID: 1, Notes: 'Εξαιρετικός γιατρός, με βοήθησε πολύ!', Rating: 5, Doctor_ID: 1, Patient_ID: 2}),

(:Review {Review_ID: 2, Notes: 'Πολύ επαγγελματίας, αλλά υπήρχε μικρή καθυστέρηση στο ραντεβού.', Rating: 4.5, Doctor_ID: 2, Patient_ID: 3}),

(:Review {Review_ID: 3, Notes: 'Κατανοητικός και με εξήγησε πλήρως τη θεραπεία.', Rating: 4.8, Doctor_ID: 3, Patient_ID: 4}),

(:Review {Review_ID: 4, Notes: 'Ευγενικός, αλλά δεν έμεινα απόλυτα ικανοποιημένος.', Rating: 3.5, Doctor_ID: 4, Patient_ID: 5}),

(:Review {Review_ID: 5, Notes: 'Άριστη εμπειρία, θα τον ξαναεπισκεφτώ σίγουρα.', Rating: 5, Doctor_ID: 5, Patient_ID: 3});

Ο CYPHER κώδικας για την δημιουργία σχέσεων Relationships

```
// Σχέση Doctor -> Appointment
MATCH (d:Doctor {Doctor_ID: 1}), (a:Appointment {Appointment_ID: 1})
CREATE (d)-[:HAS_APPOINTMENT]->(a);

MATCH (d:Doctor {Doctor_ID: 2}), (a:Appointment {Appointment_ID: 2})
CREATE (d)-[:HAS_APPOINTMENT]->(a);

MATCH (d:Doctor {Doctor_ID: 3}), (a:Appointment {Appointment_ID: 3})
CREATE (d)-[:HAS_APPOINTMENT]->(a);

MATCH (d:Doctor {Doctor_ID: 4}), (a:Appointment {Appointment_ID: 4})
CREATE (d)-[:HAS_APPOINTMENT]->(a);

MATCH (d:Doctor {Doctor_ID: 5}), (a:Appointment {Appointment_ID: 5})
CREATE (d)-[:HAS_APPOINTMENT]->(a);

// Σχέση Patient -> Appointment
MATCH (p:Patient {Patient_ID: 2}), (a:Appointment {Appointment_ID: 1})
CREATE (p)-[:BOOKED]->(a);

MATCH (p:Patient {Patient_ID: 3}), (a:Appointment {Appointment_ID: 2})
CREATE (p)-[:BOOKED]->(a);

MATCH (p:Patient {Patient_ID: 4}), (a:Appointment {Appointment_ID: 3})
CREATE (p)-[:BOOKED]->(a);

MATCH (p:Patient {Patient_ID: 5}), (a:Appointment {Appointment_ID: 4})
CREATE (p)-[:BOOKED]->(a);

MATCH (p:Patient {Patient_ID: 3}), (a:Appointment {Appointment_ID: 5})
CREATE (p)-[:BOOKED]->(a);
```

```
// Σχέση Appointment -> Payment
MATCH (a:Appointment {Appointment_ID: 1}), (pay:Payment {Payment_ID: 1})
CREATE (a)-[:HAS_PAYMENT]->(pay);

MATCH (a:Appointment {Appointment_ID: 2}), (pay:Payment {Payment_ID: 2})
CREATE (a)-[:HAS_PAYMENT]->(pay);

MATCH (a:Appointment {Appointment_ID: 3}), (pay:Payment {Payment_ID: 3})
CREATE (a)-[:HAS_PAYMENT]->(pay);

MATCH (a:Appointment {Appointment_ID: 4}), (pay:Payment {Payment_ID: 4})
CREATE (a)-[:HAS_PAYMENT]->(pay);

MATCH (a:Appointment {Appointment_ID: 5}), (pay:Payment {Payment_ID: 5})
CREATE (a)-[:HAS_PAYMENT]->(pay);

// Σχέση Patient -> MedicalHistory
MATCH (p:Patient {Patient_ID: 2}), (mh:MedicalHistory {History_ID: 1})
CREATE (p)-[:HAS_HISTORY]->(mh);

MATCH (p:Patient {Patient_ID: 3}), (mh:MedicalHistory {History_ID: 2})
CREATE (p)-[:HAS_HISTORY]->(mh);

MATCH (p:Patient {Patient_ID: 4}), (mh:MedicalHistory {History_ID: 3})
CREATE (p)-[:HAS_HISTORY]->(mh);

MATCH (p:Patient {Patient_ID: 5}), (mh:MedicalHistory {History_ID: 4})
CREATE (p)-[:HAS_HISTORY]->(mh);

MATCH (p:Patient {Patient_ID: 1}), (mh:MedicalHistory {History_ID: 5})
CREATE (p)-[:HAS_HISTORY]->(mh);

// Σχέση Doctor -> Prescription
MATCH (d:Doctor {Doctor_ID: 1}), (pr:Prescription {Prescription_ID: 1})
CREATE (d)-[:PRESCRIBED]->(pr);
```

```
MATCH (d:Doctor {Doctor_ID: 2}), (pr:Prescription {Prescription_ID: 4})
CREATE (d)-[:PRESCRIBED]->(pr);

MATCH (d:Doctor {Doctor_ID: 3}), (pr:Prescription {Prescription_ID: 2})
CREATE (d)-[:PRESCRIBED]->(pr);

MATCH (d:Doctor {Doctor_ID: 4}), (pr:Prescription {Prescription_ID: 3})
CREATE (d)-[:PRESCRIBED]->(pr);

MATCH (d:Doctor {Doctor_ID: 5}), (pr:Prescription {Prescription_ID: 5})
CREATE (d)-[:PRESCRIBED]->(pr);

// Σχέση Prescription -> MedicalTest
MATCH (pr:Prescription {Prescription_ID: 1}), (mt:MedicalTest {Test_ID: 1})
CREATE (pr)-[:BASED_ON]->(mt);

MATCH (pr:Prescription {Prescription_ID: 2}), (mt:MedicalTest {Test_ID: 3})
CREATE (pr)-[:BASED_ON]->(mt);

MATCH (pr:Prescription {Prescription_ID: 3}), (mt:MedicalTest {Test_ID: 4})
CREATE (pr)-[:BASED_ON]->(mt);

MATCH (pr:Prescription {Prescription_ID: 4}), (mt:MedicalTest {Test_ID: 2})
CREATE (pr)-[:BASED_ON]->(mt);

MATCH (pr:Prescription {Prescription_ID: 5}), (mt:MedicalTest {Test_ID: 5})
CREATE (pr)-[:BASED_ON]->(mt);

// Σχέση Patient -> Condition
MATCH (p:Patient {Patient_ID: 2}), (c:Condition {Condition_ID: 1})
CREATE (p)-[:SUFFERS_FROM]->(c);

MATCH (p:Patient {Patient_ID: 3}), (c:Condition {Condition_ID: 2})
CREATE (p)-[:SUFFERS_FROM]->(c);

MATCH (p:Patient {Patient_ID: 4}), (c:Condition {Condition_ID: 3})
CREATE (p)-[:SUFFERS_FROM]->(c);
```

```
MATCH (p:Patient {Patient_ID: 5}), (c:Condition {Condition_ID: 4})
CREATE (p)-[:SUFFERS_FROM]->(c);

// Σχέση Doctor -> Review
MATCH (d:Doctor {Doctor_ID: 1}), (r:Review {Review_ID: 1})
CREATE (d)-[:HAS_REVIEW]->(r);

MATCH (d:Doctor {Doctor_ID: 2}), (r:Review {Review_ID: 2})
CREATE (d)-[:HAS_REVIEW]->(r);

MATCH (d:Doctor {Doctor_ID: 3}), (r:Review {Review_ID: 3})
CREATE (d)-[:HAS_REVIEW]->(r);

MATCH (d:Doctor {Doctor_ID: 4}), (r:Review {Review_ID: 4})
CREATE (d)-[:HAS_REVIEW]->(r);

MATCH (d:Doctor {Doctor_ID: 5}), (r:Review {Review_ID: 5})
CREATE (d)-[:HAS_REVIEW]->(r);

// Σχέση Patient -> Review
MATCH (p:Patient {Patient_ID: 2}), (r:Review {Review_ID: 1})
CREATE (p)-[:WROTE_REVIEW]->(r);

MATCH (p:Patient {Patient_ID: 3}), (r:Review {Review_ID: 2})
CREATE (p)-[:WROTE_REVIEW]->(r);

MATCH (p:Patient {Patient_ID: 4}), (r:Review {Review_ID: 3})
CREATE (p)-[:WROTE_REVIEW]->(r);

MATCH (p:Patient {Patient_ID: 5}), (r:Review {Review_ID: 4})
CREATE (p)-[:WROTE_REVIEW]->(r);

MATCH (p:Patient {Patient_ID: 3}), (r:Review {Review_ID: 5})
CREATE (p)-[:WROTE_REVIEW]->(r);
```

13. Ο CYPHER κώδικας κώδικα μαζί με screenshot αποτελεσμάτων για τα ερωτήματα που απαντήθηκαν

Τα ερωτήματα και οι απαντήσεις στον πίνακα

1.Υπολογίστε τους ασθενείς που έχουν συνολικά πληρώσει πάνω από 100€.

```
MATCH (p:Patient)-[:BOOKED]->(a:Appointment)-[:HAS_PAYMENT]->(pay:Payment)
WITH p, SUM(pay.Amount) AS Total_Amount
WHERE Total_Amount > 100
RETURN p.Patient_ID, p.Name, Total_Amount
```

2.Υπολόγιστε τις πληρωμές ταξινομημένες κατά ημερομηνία και πόσο (φθίνουσα σειρά)

```
MATCH (p:Payment)
RETURN p.Payment_ID, p.Amount, p.Date
ORDER BY p.Date ASC, p.Amount DESC;
```

3.Υπολογίστε τα ονόματα των ασθενών , ταξινομημένα κατά ημερομηνία γέννησης από τους νεότερους προς τους μεγαλύτερους

```
MATCH (p:Patient)
RETURN p.Patient_ID, p.Name, p.Surname, p.Birth_Date
ORDER BY p.Birth_Date DESC;
```

4. Υπολογίστε τον μέσο όρο πληρωμών ανά μέθοδο πληρωμής

```
MATCH (p:Payment)
RETURN p.Payment_Method, AVG(p.Amount) AS Average_Amount
ORDER BY Average_Amount DESC;
```

5. Υπολογίστε το συνολικό ποσό πληρωμών που έχουν γίνει ανά ραντεβού.

```
MATCH (a:Appointment)-[:HAS_PAYMENT]->(p:Payment)
RETURN a.Appointment_ID, SUM(p.Amount) AS Total_Payments
```

6. Πόσα ραντεβού έχει κάθε γιατρός και ταξινόμησέ τα κατά φθίνουσα σειρά

```
MATCH (d:Doctor)-[:HAS_APPOINTMENT]->(a:Appointment)
RETURN d.Doctor_ID, d.Name AS Doctor_Name, COUNT(a.Appointment_ID) AS Total_Appointments
ORDER BY Total Appointments DESC
```

7.Υπολογίστε τις πληροφορίες για το ραντεβού , τον γιατρό , τον ασθενή και το ποσό πληρωμής

```
MATCH (d:Doctor)-[:HAS_APPOINTMENT]->(a:Appointment)<-[:BOOKED]-(p:Patient)
OPTIONAL MATCH (a)-[:HAS_PAYMENT]->(pay:Payment)
RETURN
    a.Appointment_ID AS Appointment_ID,
    a.Date AS Appointment_Date,
    a.Status AS Appointment_Status,
    p.Name AS Patient_Name,
    d.Name AS Doctor_Name,
    COALESCE(pay.Amount, 0) AS Payment_Amount;
```

Ο πίνακας με τα αποτελέσματα των ερωτημάτων (απαντημένα με την σειρά όπως είναι παραπάνω) που υλοποιούμε στο περιβάλλον της σχεσιακής βάσης και στο περιβάλλον της βάσης γραφοδεδομένων

Βάση γραφοδεδομένων	Σχεσιακή βάση
Table RAW p.Patient_ID: 3 · p.Name: "Kostas" · Total_Amount: 155	Patient_ID: 3 · Name: Kostas · Total_Amount: 155
p.Payment_ID · p.Amount · p.Date 4 · 60 · "2024-11-29" 3 · 100 · "2024-11-30" 5 · 80 · "2024-12-01" 1 · 50 · "2024-12-01" 2 · 75 · "2024-12-02"	Payment_ID · Amount · Date 4 · 60 · 2024-11-29 3 · 100 · 2024-11-30 5 · 80 · 2024-12-01 1 · 50 · 2024-12-01 2 · 75 · 2024-12-02
p.Patient_ID · p.Name · p.Surname · p.Birth_Date 4 · "Eleni" · "Vasileiou" · "2000-07-15" 2 · "Maria" · "Papadopoulou" · "1990-08-25" 1 · "John" · "Doe" · "1985-05-12" 5 · "Nikos" · "Andreadis" · "1982-03-10" 3 · "Kostas" · "Giannakos" · "1978-12-01"	Patient_ID · Name · Surname · Birth_Date 4 · Eleni · Vasileiou · 2000-07-15 2 · Maria · Papadopoulou · 1990-08-25 1 · John · Doe · 1985-05-12 5 · Nikos · Andreadis · 1982-03-10 3 · Kostas · Giannakos · 1978-12-01
p.Payment_Method · Average_Amount "Debit Card" · 100.0 "Cash" · 77.5 "Bank Transfer" · 60.0 "Credit Card" · 50.0	Payment_Method · Average_Amount Bank Transfer · 60 Cash · 77.5 Credit Card · 50 Debit Card · 100
a.Appointment_ID · Total_Payments 1 · 50 2 · 75 3 · 100 4 · 60 5 · 80	Appointment_ID · Total_Payments 1 · 50 2 · 75 3 · 100 4 · 60 5 · 80
d.Doctor_ID · Doctor_Name · Total_Appointments 1 · "Sophia" · 1 2 · "Dimitris" · 1 3 · "Eleni" · 1 4 · "Giorgos" · 1 5 · "Maria" · 1	Doctor_ID · Doctor_Name · Total_Appointments 1 · Sophia · 1 2 · Dimitris · 1 3 · Eleni · 1 4 · Giorgos · 1 5 · Maria · 1
Appointment_ID · Appointment_Date · Appointment_Status · Patient_Name · Doctor_Name · Payment_Amount 1 · "2024-12-10" · "Scheduled" · "Maria" · "Sophia" · 50 2 · "2024-12-05" · "Completed" · "Kostas" · "Dimitris" · 75 5 · "2024-12-20" · "Scheduled" · "Kostas" · "Maria" · 80 3 · "2024-12-15" · "Scheduled" · "Eleni" · "Eleni" · 100 4 · "2024-12-07" · "Cancelled" · "Nikos" · "Giorgos" · 60	Appointment_ID · Date · Status · Patient_Name · Doctor_Name · Payment_Amount 1 · 2024-12-10 · Scheduled · Maria · Sophia · 50 2 · 2024-12-05 · Completed · Kostas · Dimitris · 75 3 · 2024-12-15 · Scheduled · Eleni · Eleni · 100 4 · 2024-12-07 · Cancelled · Nikos · Giorgos · 60 5 · 2024-12-20 · Scheduled · Kostas · Maria · 80

14. Συμπέρασμα

Η παρούσα εργασία είχε ως στόχο τη δημιουργία μιας ολοκληρωμένης βάσης δεδομένων για τη διαχείριση δεδομένων ενός οργανισμού που δραστηριοποιείται στον τομέα της Υγείας. Η ανάπτυξη της βάσης δεδομένων πραγματοποιήθηκε με βάση την κατανόηση των πραγματικών αναγκών και διαδικασιών του οργανισμού, ενώ στη συνέχεια επεκτάθηκε σε μια βάση δεδομένων γράφου για την αποδοτικότερη απάντηση σε συγκεκριμένα ερωτήματα.

Βήματα Υλοποίησης

Απαιτήσεις της Εφαρμογής

Καταγράψαμε τις βασικές λειτουργικές απαιτήσεις της εφαρμογής, οι οποίες περιλάμβαναν τη διαχείριση ασθενών, γιατρών, ραντεβού, συνταγογραφήσεων, πληρωμών, ιατρικών εξετάσεων και αξιολογήσεων. Παράλληλα, περιγράψαμε τα δεδομένα που χρειαζόταν να αποθηκεύονται, καθώς και τις σχέσεις μεταξύ αυτών. Για παράδειγμα, οι ασθενείς σχετίζονταν με τα ραντεβού, τις ιατρικές εξετάσεις, και τις συνταγές, ενώ οι γιατροί συνδέονταν με τις ειδικότητές τους, τα ραντεβού και τις αξιολογήσεις τους.

Σχεδίαση Βάσης Δεδομένων

Σχεδιάσαμε το διάγραμμα Οντοτήτων-Συσχετίσεων (ERD), το οποίο αποτύπωνε λεπτομερώς όλες τις οντότητες, τα χαρακτηριστικά τους και τις συσχετίσεις μεταξύ τους. Με βάση το ERD, μετατρέψαμε τη βάση δεδομένων σε σχεσιακό μοντέλο, διασφαλίζοντας την κανονικοποίηση των πινάκων για τη μείωση του πλεονασμού δεδομένων και την εξασφάλιση της ακεραιότητας των δεδομένων.

Υλοποίηση Βάσης Δεδομένων

Η βάση δεδομένων υλοποιήθηκε στο XAMPP, με τη δημιουργία πινάκων, περιορισμών και συνδέσεων ξένων κλειδιών για την υλοποίηση των σχέσεων. Εισάγαμε δεδομένα στους πίνακες, όπως πληροφορίες ασθενών, γιατρών, ραντεβού, συνταγών, ιατρικών εξετάσεων και πληρωμών. Τα δεδομένα επιλέχθηκαν ώστε να προσομοιώνουν ρεαλιστικά σενάρια, καλύπτοντας ένα ευρύ φάσμα λειτουργιών.

Ερωτήματα SQL

Αναπτύξαμε SQL ερωτήματα για την κάλυψη των απαιτήσεων του χρήστη. Αυτά περιλάμβαναν, μεταξύ άλλων:

- Τη λίστα ασθενών και τα βασικά τους στοιχεία.
- Όλες τις συνταγές που έχουν γραφτεί για συγκεκριμένους ασθενείς.
- Τις πληρωμές που έχουν ολοκληρωθεί.
- Τη λίστα γιατρών με τις ειδικότητές τους και τα στοιχεία επικοινωνίας τους.
- Όλες τις εξετάσεις που πραγματοποιήθηκαν για έναν συγκεκριμένο ασθενή.
- Τα ραντεβού ενός γιατρού σε συγκεκριμένη ημερομηνία.

- Τους ασθενείς που επισκέφθηκαν συγκεκριμένο γιατρό.

Εφαρμογή Επικοινωνίας με τη Βάση Δεδομένων

Η εφαρμογή αναπτύχθηκε ως web-based σύστημα χρησιμοποιώντας PHP, με διασύνδεση που επιτρέπει την εύκολη πρόσβαση και διαχείριση των δεδομένων. Οι χρήστες μπορούσαν να εκτελούν ερωτήματα μέσω ενός φιλικού περιβάλλοντος και να βλέπουν τα αποτελέσματα σε πραγματικό χρόνο.

Μετατροπή σε Βάση Δεδομένων Γράφου

Μετατρέψαμε τη σχεσιακή βάση δεδομένων σε βάση δεδομένων γράφου στο Neo4j, διατηρώντας τις σχέσεις και τις συνδέσεις των δεδομένων μέσω κόμβων και ακμών. Η προσέγγιση αυτή παρείχε δυνατότητες πιο αποδοτικής αναζήτησης για σύνθετα ερωτήματα που αφορούν πολλαπλές συνδέσεις, όπως η εύρεση γιατρών που συνεργάζονται με συγκεκριμένους ασθενείς.

Προκλήσεις και Επίλυση

Κατά τη διάρκεια της υλοποίησης, αντιμετωπίσαμε προκλήσεις όπως η σωστή κανονικοποίηση της βάσης, η διαχείριση πολύπλοκων σχέσεων και η διασφάλιση της συνέπειας των δεδομένων κατά τη μετατροπή σε βάση γράφου. Με την εφαρμογή βέλτιστων πρακτικών και τη χρήση εργαλείων όπως το Neo4j, καταφέραμε να υπερβούμε τα εμπόδια και να διασφαλίσουμε την ακρίβεια και την αποτελεσματικότητα της βάσης.

Συνολική Αξιολόγηση

Η εργασία αυτή αποτέλεσε μια ολοκληρωμένη προσέγγιση στη σχεδίαση και υλοποίηση βάσεων δεδομένων, συνδυάζοντας θεωρητικές γνώσεις και πρακτική εφαρμογή. Μέσα από τη διαδικασία αυτή:

- Αναδείξαμε τη σημασία της σωστής ανάλυσης των απαιτήσεων.
- Κατανοήσαμε τη σημασία της κανονικοποίησης και της σωστής δομής των δεδομένων.
- Βιώσαμε τη μετάβαση από μια σχεσιακή σε μια γράφου βάση δεδομένων, αξιοποιώντας τη γλώσσα ερωτήσεων Cypher.

Η εργασία αυτή μπορεί να αποτελέσει τη βάση για την ανάπτυξη πραγματικών εφαρμογών στον τομέα της Υγείας, προσφέροντας αποτελεσματική και γρήγορη διαχείριση δεδομένων.